



EGE UNIVERSITY

UNDERGRADUATE THESIS

**WHEN DOES FEDERATED BATCHNORM HELP
UNDER DIFFERENTIAL PRIVACY?
AN EVAL-REGIME CHARACTERIZATION OF
DP-FEDBN ON MEDICAL ECG CLASSIFICATION**

Efe Deniz Bağlar

Supervisor: Prof. Dr. Hasan Bulut

Department of Computer Engineering

Submission Date: 15.06.2026

Bornova / İzmir

2026

**EGE UNIVERSITY INSTITUTE OF SCIENCE
(UNDERGRADUATE THESIS)**

**WHEN DOES FEDERATED BATCHNORM HELP
UNDER DIFFERENTIAL PRIVACY?
AN EVAL-REGIME CHARACTERIZATION OF
DP-FEDBN ON MEDICAL ECG CLASSIFICATION**

Efe Deniz Bağlar

Supervisor: Prof. Dr. Hasan Bulut

Department of Computer Engineering

Submission Date: 15.06.2026

Bornova / İzmir

2026

Undergraduate thesis titled "WHEN DOES FEDERATED BATCHNORM HELP UNDER DIFFERENTIAL PRIVACY? AN EVAL-REGIME CHARACTERIZATION OF DP-FEDBN ON MEDICAL ECG CLASSIFICATION", presented by Efe Deniz Bağlar and submitted by our institution, has been evaluated in accordance with the relevant provisions of the EU Undergraduate Education and Teaching Regulations and the EU Institute of Science Education and Teaching Directive. It has been found worthy of defense and was deemed successful unanimously/majority in the thesis defense examination held on 13.05.2026.

Jury Members:

Signature

Chairperson:

Hasan Bulut

N/A

Rapporteur Member:

N/A

N/A

Member:

N/A

N/A

EGE UNIVERSITY INSTITUTE OF SCIENCE

DECLARATION OF COMPLIANCE WITH ETHICAL RULES

In accordance with the relevant provisions of the EU Undergraduate Education and Teaching Regulations, I declare that the thesis titled "WHEN DOES FEDERATED BATCHNORM HELP UNDER DIFFERENTIAL PRIVACY? AN EVAL-REGIME CHARACTERIZATION OF DP-FEDBN ON MEDICAL ECG CLASSIFICATION", submitted as my undergraduate thesis, is my own work. I affirm that all results, documents, information, and evidence presented were obtained by me personally and within the scope of this thesis work. I acknowledge that any information and interpretations not derived from this thesis are appropriately cited and listed in the references. I confirm that I have not engaged in any behavior that violates patent or copyright laws during the preparation and writing of this thesis. I further declare that I have not submitted any part of this thesis for consideration in any other thesis at this university or any other institution. I assert that I have adhered to scientific ethical principles throughout all stages from the planning to the writing of this thesis, and I acknowledge that I will accept any legal consequences should the contrary be revealed.

15.06.2026

Efe Deniz Baęlar

ABSTRACT

WHEN DOES FEDERATED BATCHNORM HELP UNDER DIFFERENTIAL PRIVACY? AN EVAL-REGIME CHARACTERIZATION OF DP-FEDBN ON MEDICAL ECG CLASSIFICATION

Bağlar, Efe Deniz

B.Sc. in Computer Engineering

Supervisor: Prof. Dr. Hasan Bulut

June 2026, 76 Pages

Federated learning (FL) for medical artificial intelligence must reconcile two competing demands: per-site personalized performance and rigorous differential-privacy (DP) guarantees. Federated BatchNorm (FedBN) achieves the former by keeping BatchNorm statistics local to each client, but it is incompatible with differentially private stochastic gradient descent (DP-SGD) through standard libraries such as Opacus, because BatchNorm couples samples within a batch and breaks the per-sample gradient accounting that DP-SGD requires. As a result, most differentially private FL deployments fall back to BatchNorm-replacement workarounds — typically GroupNorm — that remove the very mechanism on which FedBN’s non-IID advantage depends.

This thesis formalizes and empirically characterizes **DP-FedBN**, a dual-optimizer composition that applies DP-SGD to the non-BatchNorm parameters while keeping the BatchNorm parameters and running statistics local and unprotected, since they never cross the federation boundary. The term “DP-FedBN” already appears as a single-paragraph, party-level DP baseline (Laplace noise on aggregated model updates) in ADCOL (ICML 2023); to our knowledge, this work is the first **record-level DP-SGD** composition of FedBN’s BatchNorm-locality with explicit treatment of the Opacus engineering obstacle, together with the first systematic medical empirical characterization of that composition across heterogeneity regimes and privacy budgets.

We benchmark FedBN, DP-FedAvg+GroupNorm, and DP-FedBN on MIT-BIH five-class beat classification and PTB-XL binary record classification across

six non-IID partition regimes and three privacy budgets (about forty configurations). Two findings emerge. First, the *eval-regime flip* is task-agnostic: FedBN ranks worst under pooled-central evaluation (losing by up to 0.31 macro F1) while ranking best under per-client local evaluation (winning by up to +0.72 macro F1), confirmed on both datasets. Second, DP-FedBN’s local advantage is partition-dependent: it preserves a +0.17 local-F1 edge over GroupNorm on label-skew at $\epsilon = 3$, but loses under Dirichlet skew on both datasets. We provide a heterogeneity-typed decision rubric for medical FL practitioners.

Keywords: federated learning, differential privacy, batch normalization, ECG classification, non-IID data, Opacus, medical AI.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Prof. Dr. Hasan Bulut, for his guidance and support throughout this thesis. I am also grateful to the faculty of the Department of Computer Engineering at Ege University, and to my family and friends for their encouragement.

TABLE OF CONTENTS

	<u>Page</u>
1. INTRODUCTION	1
2. FUNDAMENTAL CONCEPTS	6
3. RELATED WORK	17
4. METHODOLOGY	23
5. EXPERIMENTAL SETUP	32
6. RESULTS AND DISCUSSION	39
7. CONCLUSION AND FUTURE WORK	53
REFERENCES	57

LIST OF FIGURES

<u>Figure</u>	<u>Page</u>
Figure 5.1 Per-client class distributions for the six MIT-BIH partitions	35
Figure 6.1 Centralized training and validation curves on MIT-BIH and PTB-XL	40
Figure 6.2 Best central macro F1 across the partition $\times$ aggregation grid (MIT-BIH)	41
Figure 6.3 Central versus per-client local macro F1 under severe non-IID (MIT-BIH)	43
Figure 6.4 Differential-privacy privacy–utility frontier (MIT-BIH)	46
Figure 6.5 Cross-task eval-regime replication on MIT-BIH and PTB-XL	49

LIST OF CHARTSChartPage

NO CHARTS ARE USED IN THIS THESIS; ALL RESULTS ARE REPORTED IN TABLES (SEE, E.G., TABLE 6.1).

SYMBOLS AND ABBREVIATIONS

<u>Abbreviations</u>	<u>Descriptions</u>
AAMI	Association for the Advancement of Medical Instrumentation
AUC	Area Under the ROC Curve
BN	Batch Normalization
CNN	Convolutional Neural Network
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
ECG	Electrocardiogram
FedAvg	Federated Averaging
FedBN	Federated Batch Normalization
FedPerf	Performance-weighted Federated aggregation (baseline)
FedProx	Federated Proximal optimization
FL	Federated Learning
GN	Group Normalization (GroupNorm)
IID	Independent and Identically Distributed
MIA	Membership Inference Attack
MIT-BIH	MIT-BIH Arrhythmia Database
PTB-XL	PTB-XL Electrocardiography Database
RDP	Rényi Differential Privacy
SGD	Stochastic Gradient Descent

1. INTRODUCTION

1.1 Motivation

Modern cardiology produces an enormous volume of electrocardiogram (ECG) recordings, and deep neural networks have become remarkably effective at reading them: detecting arrhythmias, localizing ischemic events, and flagging conduction abnormalities at a level that, on narrow tasks, approaches that of expert cardiologists [25]. The clinical appeal is direct. Cardiac arrhythmias such as atrial and ventricular ectopy, and chronic conditions such as myocardial infarction and conduction disorders, are common, sometimes life-threatening, and frequently under-detected in routine care. An automated screen that triages ECGs at scale — in primary clinics, in wearable monitors, in emergency departments — could shorten the path from signal to diagnosis. Achieving that level of accuracy, however, depends on training data, and clinical ECG data is among the most tightly governed data that exists.

Patient records are protected by regulation in essentially every jurisdiction — the United States’ HIPAA, the European Union’s GDPR, and Türkiye’s KVKK among them — and by professional and ethical obligation independent of any statute. The straightforward engineering recipe for a strong ECG classifier, namely pooling recordings from many hospitals into one central training set, is precisely the operation that these protections are designed to prevent. A single institution rarely holds enough labeled data, across enough patient subpopulations and acquisition devices, to train a model that generalizes; yet the institutions that collectively do hold such data cannot legally or ethically merge it. This is the structural tension that motivates privacy-preserving distributed learning for medical artificial intelligence.

Federated learning (FL) [2], [3] is the canonical response. Instead of moving data to a central trainer, FL moves the training to the data: each participating site trains a model locally on its own records and shares only model updates — gradients or parameters — with a coordinating server, which aggregates them into a global model and broadcasts it back. Over many such rounds the global model improves as if it had seen every site’s data, while no raw patient recording ever leaves its institution. The standard aggregation rule, Federated Averaging (FedAvg) [2], simply averages the client updates weighted by their dataset sizes.

Federated learning removes the most obvious privacy exposure — raw data movement — but it does not by itself make training private. The shared model updates are functions of the local data, and a body of work has shown that they can leak it: membership-inference and gradient-inversion attacks can recover whether a particular record was in the training set, or even reconstruct individual training inputs, from the updates a client transmits [4], [31]. For data as sensitive as a patient’s ECG, “we only shared the gradients” is not a privacy guarantee. This motivates layering **differential privacy (DP)** [5] on top of federated learning.

Differential privacy gives a formal, quantitative guarantee that the output of a computation is statistically insensitive to the presence or absence of any single record. The dominant mechanism for training deep networks under DP is **DP-SGD** [5], which clips each per-sample gradient to a bounded norm and adds calibrated Gaussian noise before the optimizer step, yielding a tunable (ϵ, δ) -DP guarantee whose strength is controlled by a privacy budget ϵ . DP-SGD composes cleanly with federated learning: each client simply runs DP-SGD on its local optimizer, and because every transmitted update is already DP-protected at its source, the server’s aggregation needs no further privacy machinery [4]. In principle, then, private federated ECG classification is just “FedAvg with DP-SGD locally.”

In practice, one architectural component breaks this clean composition: **batch normalization (BN)** [6]. The difficulty is mechanical and is developed in full in Chapter 2; in brief, BN normalizes each sample using the mean and variance of the whole mini-batch, which couples the samples together and makes the notion of a *per-sample* gradient — the object DP-SGD must clip — ill-defined. Production DP libraries enforce this: Opacus [7], the standard PyTorch implementation of DP-SGD, refuses at setup time to wrap any model that contains a BatchNorm module. Its recommended remedy is to replace every BN layer with GroupNorm [8], which computes per-sample channel-group statistics and is therefore privacy-safe.

That remedy is where the tension this thesis addresses becomes sharp. Batch normalization is not only a convenience in the federated setting — it is the substrate of one of the most effective remedies for *non-IID* data, which is the defining difficulty of realistic federated learning. Client datasets in medicine are almost never identically distributed: a cardiology referral center, a general hospital, and a single-condition clinic see systematically different mixes of normal and abnormal beats. Under such heterogeneity, FedAvg drifts and under-converges. **FedBN** [9] is a simple and influential fix: it averages the convolutional and linear parameters

across clients exactly as FedAvg does, but it *excludes* the BatchNorm parameters and running statistics from aggregation, so that each client’s BN layers stay local and act as a learned, per-client adapter to that client’s own data distribution. FedBN’s entire mechanism is the BatchNorm state that the GroupNorm fix deletes. Replacing BN with GroupNorm to satisfy Opacus therefore does not merely change a normalization layer — it removes the personalization mechanism that made the federated model robust to heterogeneity in the first place.

1.2 Problem statement

The space of reasonable options under this three-way tension between BatchNorm, DP-SGD, and federated learning is small, and each corner sacrifices something. One can keep FedBN and abandon DP, buying personalization at the cost of any formal privacy guarantee. One can keep DP and abandon FedBN, applying DP-FedAvg with GroupNorm and accepting the loss of the BN-based personalization mechanism. Or one can attempt to *compose* the two by applying DP-SGD selectively — only to the parameters that actually leave the client. This third option is what the source paper, and this thesis, call **DP-FedBN**.

The idea is straightforward in principle and subtle in practice. Under the FedBN protocol the BatchNorm parameters and running statistics never cross the federation boundary: they are never aggregated and never broadcast. In the standard honest-but-curious federated threat model, a parameter that is never transmitted cannot be inferred from transmissions, and so it requires no DP protection at all. The only parameters that need DP are the non-BN (convolutional and linear) parameters that *do* cross the boundary. The principled construction, therefore, applies DP-SGD to the non-BN parameters while leaving the BN state to update locally and freely. Implementing this on top of off-the-shelf Opacus, however, is not obvious: it requires a non-trivial freeze-then-unfreeze maneuver to make Opacus’s validator accept a BN-containing model, two parallel optimizers — one DP-protected, one plain — and careful coordination of the parameter naming that Opacus introduces when it wraps the model. The central methodological problem of this thesis is to make that composition correct, library-compatible, and properly privacy-accounted, and then to ask the empirical question its title poses: **when does federated BatchNorm actually help under differential privacy?**

1.3 Contributions

This thesis makes three contributions, two empirical and one methodological. The framing is deliberately measured, and one point of prior art must be stated up

front. The term “DP-FedBN” is not introduced here: it already appears in **ADCOL** (Li, He, and Song, ICML 2023) [1], whose Appendix B.11 and Table 17 include a one-paragraph “DP-FedBN” baseline implemented by clipping and adding **Laplace** noise to communicated, aggregated model updates — that is, the *party-level* (client-level) DP recipe of Geyer et al. [4] — evaluated on five digit-recognition datasets. The contribution of this work is therefore neither the DP-FedBN concept nor a new aggregation rule nor a new privacy mechanism. To our knowledge, this is the **first record-level DP-SGD composition of FedBN’s BatchNorm-locality with explicit treatment of the Opacus engineering obstacle**, together with the first systematic medical empirical characterization of that composition across heterogeneity regimes and privacy budgets. Concretely:

1. **Methodological — a record-level DP-SGD construction of DP-FedBN, with engineering documentation.** We give a dual-optimizer construction, compatible with off-the-shelf Opacus 1.5, that applies DP-SGD per-sample gradient clipping to the non-BN parameters while keeping the BN parameters and running statistics local. Unlike the party-level Laplace baseline of ADCOL [1], this construction provides **record-level** (ϵ, δ) -DP — strictly stronger than a party-level Laplace mechanism — and we document the implementation obstacles (the ModuleValidator rejection of BatchNorm, the freeze-then-unfreeze workaround whose discussion lives only in an Opacus GitHub issue from 2022 [10], the state-dict prefix mismatch under GradSampleModule, and the incompatibility of the ExpandedWeights backend with BatchMemoryManager), and validate that the construction passes Opacus’s ModuleValidator without sacrificing FedBN’s mechanism.
2. **Empirical 1 — the eval-regime flip is task-agnostic.** On two datasets with substantially different tasks — MIT-BIH five-class beat-level classification and PTB-XL binary record-level classification — and at matched non-IID partitions, FedBN ranks *worst* under pooled-central evaluation while ranking *best* under per-client local evaluation. The central-versus-local macro-F1 gap reaches **+0.72** on MIT-BIH label-skew ($C = 2$). We argue that the choice between pooled-central and per-client local evaluation matters more than the choice of aggregation method in many realistic deployment scenarios.
3. **Empirical 2 — DP-FedBN’s local advantage is partition-dependent.** Under DP, the comparison between DP-FedBN and DP-FedAvg+GroupNorm is partition-sensitive. DP-FedBN preserves a **+0.17** local-F1 edge on MIT-BIH label-skew at $\epsilon = 3$, but *loses* under Dirichlet $\alpha = 0.1$ on both datasets (−0.008 on MIT-BIH and −0.234 on PTB-XL at $\epsilon = 3$). We give a mechanistic explanation — DP-driven activation noise

propagates to the per-client BN parameters without the cross-client averaging that smooths it for shared parameters — and translate the findings into a heterogeneity-typed decision rubric for medical FL practitioners.

A fourth observation supports the first contribution: naive DP-FedAvg on a BatchNorm-containing model is *empirically refused* by Opacus’s ModuleValidator at every finite ε , which is the concrete motivation for a non-naive composition rather than a side remark.

1.4 Organization of the thesis

The remainder of this thesis is organized as follows. **Chapter 2 (Fundamental Concepts)** develops, with the equations needed later, the background on which everything rests: ECG signals and arrhythmia classification, deep learning for ECG, federated learning and the non-IID problem, batch normalization and why it motivates FedBN, differential privacy and DP-SGD with its privacy accounting, and the precise mechanical conflict between batch normalization and DP-SGD. **Chapter 3 (Related Work)** situates DP-FedBN against prior art in privacy-preserving federated learning, differential privacy in healthcare machine learning, batch normalization in federated learning, and deep learning for ECG, and positions the contribution honestly against ADCOL [1]. **Chapter 4 (Methodology)** presents the DP-FedBN construction in full: the system and threat model, the model architecture, the record-level DP-SGD dual-optimizer construction, the handling of BatchNorm statistics under DP and FL, the privacy accounting, and a complete algorithm box. **Chapter 5 (Experimental Setup)** details the datasets and preprocessing, the non-IID partitioning scheme, the model and training configuration, the differential-privacy parameters, the evaluation regimes, and the implementation. **Chapter 6 (Results and Discussion)** reports the centralized baselines, the non-IID federated benchmark, the eval-regime flip, the differential-privacy privacy–utility frontier, and the cross-task replication, and interprets the partition-dependent DP behavior, the practitioner rubric, and the limitations. **Chapter 7 (Conclusion and Future Work)** summarizes the contributions, restates the limitations, and lays out concrete directions for extending the work.

2. FUNDAMENTAL CONCEPTS

This chapter develops the background needed for the rest of the thesis. It is deliberately self-contained: each concept that later chapters rely on is introduced here once, with the equations and definitions that the methodology and results will reference. The chapter proceeds from the application domain inward — first the

electrocardiogram and the clinical classification task, then the deep-learning models used to read it, then the federated-learning setting and its characteristic non-IID difficulty, then batch normalization and the FedBN remedy, then differential privacy and DP-SGD, and finally the precise mechanical conflict between batch normalization and DP-SGD that is the technical heart of this work.

2.1 ECG signals and arrhythmia classification

The electrocardiogram is a recording of the heart’s electrical activity, measured as a time-varying voltage between electrodes placed on the body surface. Each cardiac cycle produces a stereotyped waveform whose components are named by convention: the **P wave** (atrial depolarization), the **QRS complex** (ventricular depolarization, dominated by the sharp **R peak**), and the **T wave** (ventricular repolarization). The R peak is the most prominent landmark and is the standard reference point for segmenting a continuous recording into individual heartbeats.

A clinical ECG is typically recorded from multiple **leads** — different electrode configurations that view the heart’s electrical axis from different angles. The standard diagnostic ECG uses twelve leads; research and monitoring settings often use fewer. Different leads emphasize different pathologies, and the choice of which lead(s) to analyze is a substantive modeling decision, as Chapter 5 discusses for the single-lead PTB-XL task.

Two classification granularities appear in this thesis, and they are genuinely different problems:

- **Beat-level classification** asks, for each individual heartbeat, which rhythm class it belongs to. The reference standard is the **ANSI/AAMI EC57** recommendation [19], which groups the dozens of fine-grained annotation symbols in databases such as MIT-BIH into five clinically meaningful super-classes: **N** (normal and bundle-branch-block beats), **S** (supraventricular ectopic beats), **V** (ventricular ectopic beats), **F** (fusion beats), and **Q** (unknown or paced beats). This grouping is what makes beat classifiers comparable across studies [18], [20], [21].
- **Record-level classification** asks, for an entire ten-second recording, which diagnostic category it represents — for example, normal versus abnormal, or a finer split into diagnostic super-classes such as myocardial infarction, ST/T change, conduction disturbance, and hypertrophy, as defined in the PTB-XL dataset [22].

The clinical content of the five AAMI beat classes is worth making concrete, because it explains why the rarity of some classes is unfortunate rather than

incidental. The **N** class collects normal sinus beats together with bundle-branch-block beats, which are abundant. The **S** (supraventricular ectopic) and **V** (ventricular ectopic) classes are the diagnostically interesting middle ground: ventricular ectopy in particular can presage dangerous arrhythmias, yet it is far less common than normal conduction. The **F** (fusion) class — beats that are a hybrid of a normal and an ectopic activation — is genuinely rare, often well under one percent of beats, which is precisely the regime in which an accuracy-oriented classifier would simply learn to ignore it. The **Q** class gathers paced and otherwise unclassifiable beats. A clinically useful beat classifier must therefore perform well on the rare ectopic and fusion classes, not merely on the dominant normal class — a requirement that, as Section 2.2 makes explicit, the choice of evaluation metric must enforce.

A defining and unavoidable property of both tasks is **severe class imbalance**. In a representative beat database the normal class dominates — roughly four out of five beats are normal — while clinically critical classes such as fusion beats are vanishingly rare. A classifier that predicts “normal” for everything would achieve high accuracy and be clinically useless. This single fact dictates much of the evaluation methodology in Section 2.2 and Chapter 5: accuracy is a misleading score here, and a class-balanced metric is mandatory.

2.2 Deep learning for ECG

An ECG beat or record is, computationally, a one-dimensional time series — a sequence of voltage samples. The natural deep-learning architecture for such data is the **one-dimensional convolutional neural network (1-D CNN)**, which has become a workhorse for ECG classification and, on narrow tasks, has reached expert-level performance [25], [26], [27].

A 1-D convolutional layer slides a bank of learnable filters along the time axis. For an input feature map $x \in R^{C_{\text{in}} \times L}$ with C_{in} channels and length L , a convolution with kernel $w \in R^{C_{\text{out}} \times C_{\text{in}} \times k}$ produces, for output channel o and position t ,

$$(x * w)_{o,t} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{j=1}^k w_{o,c,j} x_{c,t+j-1},$$

where k is the kernel size and b_o a bias. Stacking such layers, interleaved with pointwise nonlinearities such as the rectified linear unit $\text{ReLU}(z) = \max(0, z)$ and

with **pooling** layers that downsample along time, lets the network build a hierarchy of temporal features: early layers respond to local waveform shapes (the slope of an R peak, the width of a QRS complex), later layers to longer-range morphology. A **global average pooling** layer at the end collapses the time axis to a fixed-length vector regardless of input length, which is convenient when beats and records of different durations must be handled by one architecture. One or more fully connected layers then map this vector to class scores, and a softmax converts the scores to class probabilities,

$$p_c = \frac{\exp(z_c)}{\sum_{c'} \exp(z_{c'})},$$

with training driven by the cross-entropy loss $\ell(p, y) = -\log p_y$ for true label y .

Because of the class imbalance noted in Section 2.1, model quality in this thesis is reported with the **macro-averaged F1 score** rather than accuracy. For a single class c , with TP_c , FP_c , and FN_c counting true positives, false positives, and false negatives, precision and recall are

$$precision_c = \frac{TP_c}{TP_c + FP_c}, recall_c = \frac{TP_c}{TP_c + FN_c},$$

their harmonic mean is the per-class F1 score,

$$F1_c = \frac{2precision_c \cdot recall_c}{precision_c + recall_c},$$

and the **macro F1** averages the per-class scores with equal weight,

$$\text{macro-F1} = \frac{1}{|C|} \sum_{c \in C} F1_c.$$

The equal weighting is the crucial property: a rare but clinically important class contributes as much to macro F1 as the dominant normal class, so a model cannot score well by ignoring minorities. A complementary, threshold-independent measure is the **macro-averaged area under the receiver-operating-characteristic curve (AUC)**, computed one-versus-rest and averaged over classes; it summarizes ranking quality across all decision thresholds and is reported alongside macro F1 in this work as a secondary metric. Class imbalance is further

mitigated during training by **inverse-frequency class weighting** of the cross-entropy loss, which up-weights the gradient contribution of under-represented classes so that a few abundant classes do not dominate the loss; the exact weighting used here is described in Chapter 5.

Two further practical levers shape how well such a network trains and generalizes, both used in this thesis. **Regularization** — weight decay, and dropout, which randomly zeros a fraction of activations during training — discourages overfitting, which is a real risk on the rare classes where only a handful of examples exist. **Data augmentation** — small, label-preserving perturbations of the input signal — enlarges the effective training set and is used selectively here for the harder of the two record-level tasks. The interaction of these levers with the federated and differentially private settings is non-trivial, because both federation and DP noise already act as implicit regularizers, and Chapter 6 reports where additional capacity or augmentation did and did not help.

2.3 Federated learning and the non-IID problem

Federated learning [2], [16] trains a shared model across K clients without centralizing their data. Client k holds a local dataset D_k of size $n_k = |D_k|$, and the total dataset size is $n = \sum_{k=1}^K n_k$. The learning objective is the size-weighted average of the clients' local objectives,

$$\min_{\theta} F(\theta) = \sum_{k=1}^K \frac{n_k}{n} F_k(\theta), F_k(\theta) = \frac{1}{n_k} \sum_{(x,y) \in D_k} \ell(f_{\theta}(x), y),$$

which is exactly the objective that pooled central training would minimize, but it must be optimized without ever moving the data.

Federated Averaging (FedAvg) [2] is the canonical algorithm. Training proceeds in **rounds**. In round t , the server broadcasts the current global parameters θ_t ; each client initializes its local model to θ_t , performs E epochs of local stochastic gradient descent on D_k to obtain updated parameters $\theta_t^{(k)}$, and returns them to the server; the server aggregates by a size-weighted average,

$$\theta_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \theta_t^{(k)},$$

and the next round begins. Performing several local epochs per round, rather than communicating after every gradient step, is what makes FedAvg communication-efficient.

FedAvg is provably well-behaved when the client data is **independent and identically distributed (IID)** — when every client’s data looks like a random sample from one common distribution. Real federated data almost never satisfies this. The general term for the violation is **non-IID** or **statistically heterogeneous** data, and it comes in several distinguishable forms that this thesis treats separately:

- **Label (prior) skew:** clients have different class proportions. A specialty clinic may see almost exclusively one or two conditions; a referral center sees a different mix. In the extreme, a client holds only a subset of the label space.
- **Covariate (feature) skew:** the inputs themselves are distributed differently across clients — different acquisition devices, electrode placements, or patient demographics shift the feature distribution even for the same label.
- **Quantity skew:** clients simply hold very different *amounts* of data, independent of class proportions.

Under heterogeneity, the local optima $\theta_t^{(k)}$ that the clients reach in a round can point in conflicting directions; their average can be a poor compromise that fits no client well. This phenomenon, **client drift**, makes FedAvg converge more slowly and to a worse solution than centralized training, and the effect worsens as heterogeneity increases. The mechanism is easy to state: each client minimizes its *own* objective F_k , and when the F_k are pulling toward different minima, several local SGD steps move each client toward its local minimum, so the average of the resulting parameters need not decrease the *global* objective F at all. The more local computation per round — the larger E — the more pronounced the drift, which is the central tension of communication-efficient FL: fewer rounds are cheaper but amplify the disagreement between clients.

This drift also exposes a subtlety that becomes the empirical centerpiece of this thesis. The global objective $F(\theta)$ measures performance on the *pooled* mixture of all clients’ data, but in a heterogeneous deployment each client ultimately runs the model on its *own* distribution. A model that is poor on the pooled mixture may nonetheless be excellent on each client’s individual distribution, and vice versa. Whether one cares about pooled performance or per-client performance is therefore

not a detail of measurement but a choice of objective, and Chapter 6 shows that the choice can reverse the ranking of methods. Two influential responses to client drift frame the rest of this thesis.

FedProx [11] keeps FedAvg’s aggregation but modifies the local objective, adding a proximal term that anchors each client’s local solution to the broadcast global model:

$$\min_{\theta} F_k(\theta) + \frac{\mu}{2} \|\theta - \theta_t\|_2^2.$$

The penalty discourages any client from moving too far from the consensus in a single round, trading some local fit for stability. The strength μ is a hyperparameter; an un-tuned μ can under-perform plain FedAvg, a point Chapter 6 returns to.

FedBN [9] takes an orthogonal route that is central to this thesis and is developed in Section 2.4: rather than changing the objective, it changes *which parameters are aggregated*, leaving each client’s batch-normalization state out of the average so that it can specialize to that client’s distribution.

The construction of controlled non-IID partitions for experiments — including the widely used Dirichlet- α scheme [28], [29] — is described in Chapter 5.

2.4 Batch normalization and why it is special in federated learning

Batch normalization (BN) [6] is a layer inserted between a linear/convolutional operation and its nonlinearity that standardizes activations using statistics computed over the current mini-batch. For a mini-batch B of pre-activations $\{x_i\}_{i \in B}$ at a given feature (channel), BN first computes the batch mean and variance,

$$\mu_B = \frac{1}{|B|} \sum_{i \in B} x_i, \sigma_B^2 = \frac{1}{|B|} \sum_{i \in B} (x_i - \mu_B)^2,$$

then normalizes and applies a learnable affine transformation with scale γ and shift β ,

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon_{\text{BN}}}}, y_i = \gamma \hat{x}_i + \beta,$$

where ϵ_{BN} is a small constant for numerical stability. Normalizing activations in this way stabilizes and accelerates training. Crucially, BN behaves differently at training and inference time. During training it uses the batch statistics (μ_B, σ_B^2) above. For inference, where a single example has no batch to normalize against, BN uses **running statistics** (μ, σ^2) accumulated during training as exponential moving averages,

$$\mu \leftarrow (1 - m)\mu + m\mu_B, \sigma^2 \leftarrow (1 - m)\sigma^2 + m\sigma_B^2,$$

with momentum m . A BN layer therefore carries two kinds of state: the **learnable affine parameters** (γ, β) , which are updated by gradient descent, and the **running buffers** (μ, σ^2) , which are *not* gradient-updated but are accumulated by the forward pass. Both kinds of state encode information about the distribution of activations the layer has seen.

This is exactly what makes BN special in the federated setting. The running statistics of a client’s BN layers summarize that client’s data distribution. If, under heterogeneity, two clients see very different data, their BN statistics diverge to reflect it. FedAvg blindly averages all parameters and buffers, including BN state, which forces every client to normalize using a single global compromise that fits none of them — a particularly damaging form of client drift.

FedBN [9] is the remedy. It partitions the model parameters into a BatchNorm group and a non-BatchNorm group, aggregates the non-BatchNorm parameters by the ordinary FedAvg rule, and **excludes the BatchNorm parameters and running statistics from aggregation entirely**. Each client thus keeps its own BN affine parameters (γ, β) and its own running buffers (μ, σ^2) across all rounds; the BN layers act as a learned, per-client adapter that calibrates the shared convolutional features to the local data distribution. FedBN was originally validated on feature-shifted image partitions, where it improved over FedAvg. Its mechanism — keeping BatchNorm state local — is both the source of its non-IID advantage and, as Section 2.6 and Chapter 4 show, the source of its incompatibility with off-the-shelf DP-SGD.

2.5 Differential privacy and DP-SGD

Differential privacy (DP) [5], [23] is a formal, quantitative definition of what it means for an algorithm to protect individual records. The intuition is a stability requirement: the output distribution of the algorithm should be almost unchanged if any one record is added to or removed from the input. Two datasets are called **adjacent** if they differ in exactly one record. A randomized mechanism M satisfies (ϵ, δ) -**differential privacy** if, for every pair of adjacent datasets D and D' and every measurable set S of possible outputs,

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta.$$

The **privacy budget** $\epsilon \geq 0$ bounds how much the presence of a single record can shift the output distribution: smaller ϵ means stronger privacy. The parameter δ is a small probability of failing the ϵ bound and is conventionally set far below $1/n$. A central feature of DP is **composition**: the privacy cost of a sequence of DP computations accumulates in a controlled way, which is what allows a guarantee for an iterative training procedure made of many noisy steps.

The concrete threats that this guarantee defends against are not hypothetical. **Membership-inference attacks** can determine, from a trained model or its updates, whether a particular individual’s record was in the training data [30] — already a serious disclosure when the dataset is, for example, a cohort of cardiac patients. **Gradient-inversion (or “deep leakage”) attacks** go further and reconstruct individual training inputs from the gradients a client transmits [31], which is directly relevant to federated learning because gradients or parameter updates are exactly what clients share. Differential privacy is valuable precisely because its guarantee is *worst-case and attack-agnostic*: it bounds what any adversary, present or future, with any side information, can learn about any single record, rather than defending against a specific known attack. This is the property that makes it the standard for releasing models trained on sensitive medical data.

The basic tool for releasing a real-valued function under DP is the **Gaussian mechanism**. If a function g has ℓ_2 -**sensitivity** $\Delta = \max_{D \sim D'} \|g(D) - g(D')\|_2$ — the most that one record can change g — then releasing $g(D) + N(0, \sigma^2 \Delta^2 I)$ satisfies DP with a strength governed by the noise scale σ . Larger noise buys smaller ϵ . The art of training under DP is to bound the sensitivity of a gradient step and calibrate the noise accordingly.

Differentially private SGD (DP-SGD) [5] does exactly this for stochastic gradient descent. The obstacle is that an ordinary mini-batch gradient has unbounded sensitivity: a single pathological example can make the gradient arbitrarily large, so no fixed amount of Gaussian noise would suffice. DP-SGD bounds the sensitivity by **per-sample gradient clipping**. At each step, for every example x_i in the mini-batch B , it computes the per-sample gradient $g_i = \nabla_{\theta} \ell(f_{\theta}(x_i), y_i)$, clips it to a maximum ℓ_2 norm C ,

$$\hat{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right),$$

so that no single record contributes a gradient of norm greater than C , then sums the clipped per-sample gradients and adds Gaussian noise calibrated to that bounded sensitivity:

$$\hat{g} = \frac{1}{|B|} \left(\sum_{i \in B} \hat{g}_i + N(0, \sigma^2 C^2 I) \right).$$

The optimizer then takes its step using $\hat{g}$ in place of the ordinary gradient. Two ingredients are doing the work: clipping bounds the contribution of any one record to C , and the noise hides that bounded contribution. The **noise multiplier** σ controls the trade-off — more noise yields a smaller ε but a noisier, less useful gradient.

The total privacy cost of running DP-SGD for many steps is tracked by a **privacy accountant**. The tightest practical accountants are based on **Rényi differential privacy (RDP)** [24]. A mechanism is (α, ρ) -RDP if, for a Rényi-divergence order $\alpha > 1$, the divergence between its output distributions on adjacent datasets is at most ρ ; RDP composes by simply *adding* the ρ values across steps, and a final conversion turns the accumulated RDP guarantee into a standard (ε, δ) -DP guarantee. Because each DP-SGD step operates on a randomly sampled mini-batch, the **privacy amplification by subsampling** further reduces the cost: a record that is not sampled in a given step incurs no privacy loss that step. The accountant takes the sampling rate $q = |B|/n$, the noise multiplier σ , and the number of steps T , and returns the spent ε . In practice one inverts this: given a target (ε, δ) , the number of steps, and the sampling rate, the accountant solves for the smallest noise multiplier σ that meets the budget. The **Opacus** library [7] implements all of this — per-sample gradient computation, clipping, noise addition, and an RDP

accountant — behind a PrivacyEngine that wraps an ordinary PyTorch model and optimizer; its `make_private_with_epsilon` entry point performs exactly the target- (ϵ, δ) -to- σ calibration just described.

2.5.1 The granularity of privacy: record-level and client-level DP

A point that is easy to overlook, but that distinguishes this work from its closest prior art, is *what counts as one record* — that is, the **unit of adjacency** in the DP definition. Two granularities are standard in federated learning, and they protect different things.

Under **record-level (also example-level) DP**, two datasets are adjacent if they differ in a single training example — one heartbeat or one ten-second recording. A record-level guarantee bounds what an adversary can learn about any individual record. DP-SGD with **per-sample** gradient clipping, as developed above, provides exactly this: clipping the gradient of each example bounds the influence of that example.

Under **client-level (also party-level) DP**, two datasets are adjacent if they differ in one client’s *entire* contribution — all of that institution’s records at once. A client-level guarantee bounds what can be learned about whether a whole institution participated. The canonical mechanism, due to Geyer et al. [4] and McMahan et al. [32], clips and noises the *aggregated client update* rather than per-sample gradients: each client’s whole update is treated as the sensitive object whose sensitivity is bounded.

These guarantees protect different units and are calibrated by different mechanisms, so they are not interchangeable; which one is appropriate depends on whether the privacy concern is the individual patient record or the participation of a whole site. The distinction is central to the positioning of this thesis. The “DP-FedBN” baseline that appears in the prior literature is a **client-level Laplace** mechanism on aggregated updates (Chapter 3), whereas the construction developed in Chapter 4 applies **record-level** DP-SGD via per-sample gradient clipping. Stating which granularity a method provides is therefore not pedantry; it is the difference between two genuinely distinct privacy claims.

2.6 The conflict between batch normalization and DP-SGD

The two preceding sections set up a collision that is the technical center of this thesis. DP-SGD’s entire guarantee rests on the existence of a well-defined **per-**

sample gradient g_i that depends on record i alone — only then does clipping g_i to norm C bound the influence of record i , and only then does the calibrated noise yield the claimed (ϵ, δ) -DP. Batch normalization violates this premise.

Recall from Section 2.4 that, under BN during training, the normalized activation for sample i is

$$\mathbf{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon_{\text{BN}}}},$$

where the batch statistics μ_B and σ_B^2 are computed over *every* sample in the mini-batch B . The output for sample i therefore depends on all the other samples $j \neq i$ through μ_B and σ_B^2 . Differentiating the loss, the “per-sample” gradient g_i inherits this dependence: it is a function not only of (x_i, y_i) but of the whole batch. Two consequences follow. First, the object DP-SGD wants to clip is not actually attributable to a single record. Second, and more damagingly for the privacy analysis, removing or replacing one record changes the batch statistics and hence changes *every other sample’s* gradient too, so per-sample clipping to norm C no longer bounds the sensitivity of one record’s contribution to the aggregate. The clean Gaussian-mechanism argument behind Equation (2.1) collapses.

This is not a subtle numerical issue that can be tuned away; it is structural, and production DP libraries treat it as a hard error. Opacus’s `ModuleValidator` inspects a model’s architecture when the privacy engine is attached and **refuses to wrap any model containing a BatchNorm module**, raising an `UnsupportedModuleError` before training can begin. The library’s recommended remedy, exposed as `ModuleValidator.fix(model)`, replaces every `BatchNorm` layer with **GroupNorm** [8]. `GroupNorm` normalizes each sample using statistics computed over a group of that sample’s own channels, with no dependence on other samples in the batch and no running buffers, so the per-sample gradient is well-defined again and DP-SGD applies cleanly. The same property is shared by the related sample-wise normalizers `LayerNorm` and `InstanceNorm`; `GroupNorm` is simply the default that Opacus substitutes, and it interpolates between the two as its group count varies. What all of these share — and what makes them DP-compatible — is the very thing that makes them useless to FedBN: none of them maintains the per-client running statistics that FedBN keeps local.

The `GroupNorm` fix is sound for differential privacy and is, in fact, the standard way to train BN-style architectures under DP-SGD. But it is precisely the

wrong fix for FedBN. FedBN’s non-IID advantage comes entirely from its per-client BatchNorm running statistics — the learnable, distribution-specific buffers that GroupNorm does not have. GroupNorm carries no running statistics, so there is no per-client BN state for FedBN’s exclusion rule to keep local, and the personalization mechanism vanishes. The remainder of this thesis is, in essence, about navigating this conflict: keeping BatchNorm — and therefore FedBN’s mechanism — alive while still obtaining a record-level (ϵ, δ) -DP guarantee, by recognizing that under FedBN the BatchNorm state never crosses the federation boundary and therefore never needs DP protection in the first place. Chapter 4 makes that construction precise.

3. RELATED WORK

This chapter situates DP-FedBN within four strands of prior work — federated aggregation under non-IID data, differential privacy in (healthcare) federated learning, batch normalization in federated learning, and deep learning for ECG — and then positions the contribution precisely against the one prior paper that already uses the name “DP-FedBN.” The goal is an honest account of what is established, what is borrowed, and what is genuinely new here, in the measured terms the contribution warrants.

3.1 Federated aggregation under non-IID data

Federated averaging (FedAvg) [2] is the foundation on which essentially all of this literature builds: clients run local SGD and the server averages their updates weighted by sample count (Chapter 2). FedAvg is communication-efficient and works well when client data is identically distributed, but it degrades under the statistical heterogeneity that defines realistic federated deployments, and a large body of work addresses that degradation. Kairouz et al. [16] survey the field comprehensively and catalogue non-IID data as one of its central open problems.

The responses divide broadly into methods that change the *optimization* and methods that change the *model* or its *aggregation*. **FedProx** [11] is the canonical optimization-side response: it adds a proximal term to each client’s local objective that anchors the local solution to the broadcast global model, damping client drift at the cost of some local fit. The strength of the proximal term is a hyperparameter, and a poorly chosen value can make FedProx under-perform plain FedAvg — a sensitivity that this thesis observes directly. On the aggregation side, **FedBN** [9] leaves FedAvg’s averaging rule unchanged for convolutional and linear parameters

but excludes batch-normalization parameters and running statistics from the average, so that each client’s BN layers calibrate to its own data distribution and act as a learned per-client domain adapter. FedBN was originally validated on feature-shifted image partitions (the DomainNet and Office–Caltech benchmarks), where keeping BN local improved over FedAvg.

A broader family of **personalized federated learning** methods shares FedBN’s goal of producing models adapted to each client rather than a single global compromise [16]. These methods fall into a few recognizable categories: *clustered* approaches that group similar clients and learn one model per cluster; *meta-learning* and *multi-task* approaches that learn a shared initialization or related per-client models; and *parameter-decoupling* approaches that share some layers globally while keeping others local. FedBN belongs squarely to the last category — it is one of the simplest possible parameter-decoupling rules, sharing everything except the batch-normalization layers — and its appeal is precisely that simplicity: it adds no new objective, no clustering step, and no second optimization loop. Several extensions build directly on FedBN’s normalization idea, including batch-specific batch normalization combined with sharpness-aware minimization for privacy-preserving brain–computer interfaces [12], and personalized data transformations learned per client [13]. This thesis treats plain FedBN as its personalization baseline and does not deploy these extensions, both to keep the comparison clean and because the questions it asks — about evaluation regime and about composition with differential privacy — are orthogonal to which specific personalization rule is used. A simple performance-weighted aggregation baseline, here called FedPerf, is also included to contextualize FedBN; it is not claimed as a contribution and is described in Chapter 5.

3.2 Differential privacy in federated and healthcare machine learning

Differential privacy entered deep learning with **DP-SGD** [5], which clips per-sample gradients to a bounded norm and adds calibrated Gaussian noise (Chapter 2). Its composition with federated learning is direct: each client runs DP-SGD on its local optimizer, and because every transmitted update is already differentially private at its source, the server’s aggregation requires no further privacy mechanism [4]. The **Opacus** library [7] provides the standard PyTorch implementation, including a PrivacyEngine, an RDP-based privacy accountant, the ModuleValidator architectural-compatibility checker, and memory-management utilities for per-sample gradient computation; this work uses Opacus 1.5 throughout.

A crucial axis within DP for federated learning is the **granularity** of the guarantee, introduced in Section 2.5.1. **Record-level** DP, the natural product of DP-SGD with per-sample clipping, protects each individual training record. **Client-level (party-level)** DP, due to Geyer et al. [4] and McMahan et al. [32], instead clips and noises the entire aggregated client update, protecting whether a whole institution participated. The two protect different units and are calibrated by different mechanisms; both are legitimate, and the appropriate choice depends on whether the privacy concern is the individual patient record or the participating site. This distinction is the hinge on which the present contribution turns, and Section 3.5 returns to it.

Differential privacy is not the only privacy technology used in federated learning, and it is useful to place it among its neighbors. **Secure aggregation** [33] is a complementary cryptographic protocol that lets the server compute the *sum* of client updates without seeing any individual update; it hides each client’s contribution from the server but, unlike DP, provides no guarantee about what the *aggregate* (or the final model) reveals about individual records. The two are routinely combined: secure aggregation protects updates in transit, while differential privacy bounds what the released model leaks. This thesis is concerned with the latter guarantee. On the accounting side, the privacy analysis of DP-SGD has itself matured over time, from the original moments accountant of Abadi et al. [5] to the tighter Rényi differential privacy accounting [24] that Opacus uses by default; tighter accounting directly translates into less noise for the same (ϵ, δ) budget, and hence better utility. This matters acutely in medicine: medical datasets are often small and dominated by a few classes, so the rare-but-important classes — exactly the ones macro F1 is designed to reward — are the first casualties of DP noise. The utility cost of privacy is therefore not uniform across classes, which is part of why the privacy–utility trade-off in Chapter 6 is reported with class-balanced metrics under both evaluation regimes.

In healthcare specifically, federated learning has been advocated as the natural paradigm for multi-institutional medical AI precisely because it avoids centralizing protected patient data [3]. Layering differential privacy on top is widely regarded as necessary rather than optional in this setting, given that shared updates can leak training data (Section 2.5). Within medical federated learning under DP and non-IID conditions, the most directly comparable benchmark is **FedOnco-Bench** [15], which provides a careful, reproducible evaluation of privacy-aware federated tumor segmentation under heterogeneity. Notably, FedOnco-Bench

evaluates FedBN and DP-SGD as *competing alternatives* — one or the other — rather than as a single combined method, which is exactly the composition this thesis constructs and characterizes. The orthogonal question this thesis surfaces — whether one should report pooled-central or per-client local performance — is, we believe, of independent interest to this medical-FL benchmarking community.

3.3 Batch normalization in federated learning and existing DP workarounds

The incompatibility between batch normalization and DP-SGD is mechanical and unavoidable (Section 2.6): BN couples samples within a batch through shared statistics, which makes the per-sample gradient that DP-SGD must clip ill-defined, and Opacus’s `ModuleValidator` consequently rejects any model containing a `BatchNorm` module. The standard workaround, `ModuleValidator.fix()`, replaces every BN layer with **GroupNorm** [8], whose per-sample channel-group statistics are DP-safe. This is sound for differential privacy and is the conventional way to train normalization-heavy architectures under DP-SGD.

The difficulty, for the present setting, is that this workaround is incompatible with FedBN’s design. `GroupNorm` carries no learnable running statistics, so there is no per-client BN state for FedBN’s exclusion rule to keep local — the very mechanism that gives FedBN its non-IID advantage is gone. Two recent works address the broader BN-under-DP gap differently. **ResNetFed** [14] modifies the network architecture globally to support differentially private federation for privacy-preserving pneumonia detection from chest radiographs, at the cost of replacing batch normalization throughout the network. **FedOnco-Bench** [15], as noted above, benchmarks FedBN and DP-SGD side by side as alternatives rather than composing them. Neither keeps `BatchNorm` alive *and* obtains a record-level DP-SGD guarantee at the same time; the construction in Chapter 4 does, by exploiting the fact that under FedBN the `BatchNorm` state never crosses the federation boundary and therefore never needs DP protection.

It is worth being explicit about why the obvious alternatives are unsatisfying for a deployment that genuinely needs both personalization and privacy. Abandoning DP keeps FedBN’s advantage but forfeits any formal privacy guarantee. Abandoning FedBN — using DP-FedAvg with the `GroupNorm` fix — keeps the privacy guarantee but discards the personalization mechanism. Replacing BN with `GroupNorm` globally, as **ResNetFed** does, changes the architecture and again removes the per-client normalization state. The remaining option, and the one

this thesis pursues, is to compose the two by protecting only what crosses the federation boundary.

3.4 Deep learning for ECG and arrhythmia classification

The application domain has its own mature literature. The **MIT-BIH Arrhythmia Database** [18] is the long-standing reference benchmark for beat-level classification, and the **ANSI/AAMI EC57** standard [19] defines the five-class beat grouping and the inter-patient evaluation conventions that make results comparable across studies; classical and modern approaches alike — from morphology-and-interval feature engineering [20] to ensembles fusing temporal and morphological information [21] — are calibrated against it. Deep learning has pushed beat- and rhythm-classification performance substantially: convolutional networks trained on large single-lead datasets have reached cardiologist-level performance on rhythm classification [25], and transferable convolutional representations have become a standard recipe for MIT-BIH beat classification [26].

A recurring methodological theme in this literature, established by de Chazal et al. [20], is the distinction between *intra-patient* and *inter-patient* evaluation. Under intra-patient evaluation, beats from one patient may appear in both the training and the test set; under the stricter inter-patient paradigm, the patient populations are disjoint, which is more representative of deployment on previously unseen patients and typically yields lower, more honest scores. The axis this thesis investigates — pooled-central versus per-client local evaluation — is orthogonal to the intra/inter-patient axis, and the partitioning and test-set construction it uses are described and qualified in Chapter 5.

For record-level classification, the **PTB-XL** dataset [22] is the largest publicly available clinical 12-lead ECG corpus, with standardized diagnostic super-class labels and canonical stratified folds that fix the train/test split across studies. Strodthoff et al. [27] provide the reference deep-learning benchmark on PTB-XL, characterizing how convolutional and recurrent architectures perform across its diagnostic tasks. These ECG benchmarks are, however, almost entirely *centralized*: they assume the data is pooled. The contribution of this thesis is not a new ECG architecture or a new benchmark score — the model used here is a deliberately standard BN-heavy 1-D CNN — but a study of how federation, personalization, and differential privacy interact on these two well-understood tasks.

3.5 Positioning: DP-FedBN and ADCOL (ICML 2023)

The name “DP-FedBN” is not introduced by the source paper or by this thesis. It already appears in **ADCOL** — *Adversarial Collaborative Learning on non-IID features* (Li, He, and Song, ICML 2023) [1] — whose Appendix B.11 and Table 17 include a one-paragraph “DP-FedBN” baseline. That baseline is implemented by clipping and adding **Laplace** noise to communicated, aggregated model updates — that is, the **party-level (client-level)** DP recipe of Geyer et al. [4] — and is evaluated on five digit-recognition datasets at privacy budgets $\epsilon \in \{2, 5, 10\}$. Stating this clearly is part of calibrating the present claim honestly: DP-FedBN is a prior name, and the dual-optimizer composition idea is not claimed here as a conceptual invention.

The contribution of this work is therefore deliberately narrow and specific. To our knowledge, this is the **first record-level DP-SGD composition of FedBN’s BatchNorm-locality with explicit treatment of the Opacus engineering obstacle**, together with the first systematic medical empirical characterization of that composition across heterogeneity regimes and privacy budgets. Three concrete differences distinguish it from the ADCOL baseline:

1. **Privacy granularity.** This work applies DP-SGD via *per-sample gradient clipping*, yielding a **record-level** (ϵ, δ) -DP guarantee, which is a stronger guarantee than the party-level Laplace mechanism of the ADCOL baseline in the comparison drawn by the source paper. The two operate at different units of adjacency (Section 2.5.1).
2. **Engineering.** This work documents the concrete obstacle that makes the composition non-trivial in practice — Opacus’s ModuleValidator rejection of BatchNorm — and packages a freeze-then-unfreeze workaround whose enabling discussion exists only in an Opacus GitHub issue from 2022 [10], along with the further pitfalls (state-dict key prefixing under GradSampleModule, and the incompatibility of the ExpandedWeights backend with the batch-memory manager) detailed in Chapter 4. The ADCOL baseline, being a party-level Laplace mechanism on aggregated updates, never encounters this per-sample machinery and so never confronts the obstacle.
3. **Empirical scope.** This work characterizes the composition on **two medical ECG tasks, across six non-IID partition regimes and three privacy budgets** — about forty configurations — rather than a single label-balanced digit-recognition setting.

In summary, this thesis contributes neither a new aggregation algorithm nor a new privacy mechanism. It contributes a record-level DP-SGD *construction* of

the FedBN-plus-DP composition with documented Opacus engineering, a multi-partition, multi-budget medical evaluation of that construction on MIT-BIH and PTB-XL, and — orthogonally — the observation that the choice between pooled-central and per-client local evaluation is itself a decision that can reverse which method appears best. The following chapters develop the construction (Chapter 4), the evaluation (Chapters 5–6), and that orthogonal observation in full.

4. METHODOLOGY

This chapter presents DP-FedBN, the record-level differentially private composition of FedBN developed in this thesis. It states the system and threat model, fixes the model architecture, recalls the two preliminaries from Chapter 2 that the construction rests on, gives the FedBN aggregation rule, and then presents the core contribution: a dual-optimizer construction that applies DP-SGD to the parameters that cross the federation boundary while keeping the batch-normalization state local. The chapter closes with the privacy accounting, a complete client-side algorithm, and the implementation pitfalls that a re-implementer would otherwise rediscover the hard way.

The framing is the same measured one used throughout: this work contributes neither a new aggregation algorithm nor a new privacy mechanism. To our knowledge, it is the first record-level DP-SGD composition of FedBN’s BatchNorm-locality with explicit treatment of the Opacus engineering obstacle, together with the first systematic medical empirical characterization of that composition across heterogeneity regimes and privacy budgets. The prior “DP-FedBN” baseline of ADCOL [1] is a party-level Laplace mechanism on aggregated updates (Chapter 3); the construction here is record-level via per-sample gradient clipping.

4.1 System and threat model

We assume a federated setting with K clients indexed $k = 1, \dots, K$. Each client owns a local dataset D_k of size $n_k = |D_k|$. A global model f_θ with parameters θ is trained for R rounds; in each round, every client receives the current global parameters θ_t , performs E epochs of local training on D_k , and returns its updated parameters to the server, which aggregates them and broadcasts the result for the next round.

We adopt the standard federated-learning threat model: an **honest-but-curious server** that faithfully follows the protocol but attempts to infer client data from the parameter updates it observes, together with **possibly malicious peer clients** that may inspect the broadcasts they receive. The adversary’s view is therefore exactly the set of objects that cross the federation boundary.

This is what makes the DP-FedBN construction natural. Under DP-FedBN — as under plain FedBN — the only objects that cross the boundary are the non-BatchNorm parameters $\theta^{\overline{BN}}$ (the convolutional and linear weights and biases). The batch-normalization parameters and running statistics θ^{BN} are never aggregated and never broadcast; they remain on the client throughout. Consequently, in this threat model the only objects requiring differential-privacy protection are the $\theta^{\overline{BN}}$ updates, and those are DP-protected at every local step (Section 4.5). The end-to-end privacy guarantee on $\theta^{\overline{BN}}$ follows from Opacus’s privacy accountant applied per client. Because the batch-normalization state θ^{BN} never leaves the client, it requires no DP protection under this model — a parameter the adversary never sees cannot be inferred from what the adversary sees.

Two properties of differential privacy make this argument rigorous rather than merely intuitive. First, **post-processing invariance**: any function computed from a differentially private output is itself differentially private, with no additional privacy cost. The server’s size-weighted average of DP-protected client updates is such a function, so the aggregate inherits the clients’ guarantee without further machinery. Second, the guarantee is **worst-case over auxiliary information**: it holds regardless of what side information the honest-but-curious server or a malicious peer may hold, which is the property that makes a formal DP claim meaningful against an unknown adversary. Differential privacy is, in this setting, complementary to cryptographic protections such as secure aggregation (Section 3.2): the latter would hide individual updates in transit, while DP bounds what the released model leaks; this thesis is concerned with the latter.

We state the boundary of this model explicitly. If a stronger threat model is required — for instance, one in which the per-client batch-normalization statistics could be inferred through repeated query attacks against a *deployed* personalized model — then additional protections on the deployed model would be needed. We do not address that scenario here and note it as a limitation in Chapter 7.

4.2 Model architecture

Every experiment in this thesis uses a single architecture, a BatchNorm-heavy one-dimensional convolutional network (denoted CNN1D) with approximately 4.37×10^5 parameters. It is deliberately conventional: the contribution lies in the privacy composition and the evaluation, not in the network. The architecture is four convolutional blocks followed by a global average pool and a two-layer classifier:

- **Block 1:** Conv1D($1 \rightarrow 64$, kernel 7) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2)
- **Block 2:** Conv1D($64 \rightarrow 128$, kernel 5) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2)
- **Block 3:** Conv1D($128 \rightarrow 256$, kernel 5) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2)
- **Block 4:** Conv1D($256 \rightarrow 256$, kernel 3) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- **Head:** GlobalAveragePool $\rightarrow$ FC($256 \rightarrow 128$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ FC($128 \rightarrow \text{num_classes}$)

The global average pool makes the network agnostic to input length, so the same architecture serves both the 250-sample MIT-BIH beats and the 1000-sample PTB-XL records. The point relevant to this chapter is that the network contains **five batch-normalization modules** — one after each of the four convolutions and one after the first fully connected layer — and it is the parameters and running buffers of these five modules that the DP-FedBN construction treats specially. The set θ^{BN} collects, across all five modules, the learnable affine parameters (γ, β) and the running buffers (μ, σ^2) (together with the integer batch-count buffer); $\theta^{\overline{BN}}$ is everything else.

4.3 Preliminaries

Two facts established in Chapter 2 are load-bearing here and are recalled without re-derivation. First, **DP-SGD** [5] bounds the influence of any single record by clipping each per-sample gradient $g_i = \nabla_{\theta} \ell(f_{\theta}(x_i), y_i)$ to norm C and adding Gaussian noise, producing the privatized step of Equation (2.1),

$$\hat{g} = \frac{1}{|B|} \left(\sum_{i \in B} \hat{g}_i + N(0, \sigma^2 C^2 I) \right), \hat{g}_i = g_i \cdot \min(1, C / \|g_i\|_2),$$

whose noise multiplier σ an accountant chooses to meet a target (ϵ, δ) guarantee.

Second, the **BatchNorm obstacle**: under batch statistics μ_B, σ_B^2 , the normalized output for sample i is

$$\mathfrak{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon_{BN}}},$$

which depends on every other sample $j \neq i$ in the batch through μ_B and σ_B^2 . The per-sample gradient g_i therefore depends on samples other than i , and per-sample clipping no longer bounds the sensitivity of one record. Opacus’s ModuleValidator detects this and refuses to wrap any model containing a BatchNorm module; its standard remedy, ModuleValidator.fix(model), replaces every BatchNorm layer with GroupNorm, which is privacy-safe but removes the per-client BN buffer mechanism that FedBN relies on.

4.4 FedBN aggregation

FedBN [9] modifies the aggregation rule, not the local training. Partition the model parameters as $\theta = (\theta^{BN}, \theta^{\overline{BN}})$, where θ^{BN} collects all batch-normalization scale-shift parameters (γ, β) and running buffers (μ, σ^2) across all BN layers, and $\theta^{\overline{BN}}$ is everything else. FedBN aggregates the non-BN parameters by the ordinary size-weighted FedAvg average and leaves the BN parameters untouched on each client:

$$\theta_{t+1}^{\overline{BN}} = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} \theta_t^{(k), \overline{BN}}, \theta_{t+1}^{(k), BN} = \theta_t^{(k), BN}.$$

The BN parameters and buffers stay client-local; the rest is averaged exactly as in FedAvg. Because the BN state never leaves the client, in the federated threat model of Section 4.1 it requires no DP protection.

4.5 The record-level DP-SGD construction

The challenge in composing FedBN with DP-SGD is implementational rather than conceptual. Opacus’s ModuleValidator checks the model architecture *eagerly* when PrivacyEngine.make_private_with_epsilon is called, and refuses to proceed if the model contains BatchNorm. The construction must therefore make Opacus

accept a BatchNorm-containing model while still privatizing exactly the parameters that cross the boundary, and no others.

We resolve this by **temporarily freezing the BatchNorm parameters** so that the validator treats them as inert, non-trainable modules and proceeds. Concretely, before the privacy engine is attached, each BatchNorm module’s weight and bias have `requires_grad` set to `False`. This is the crux of the construction, and it works because of a detail of how Opacus inspects a model: the `ModuleValidator` iterates only over the *trainable* modules, and `GradSampleModule` registers its per-sample-gradient hooks only on modules that own trainable parameters. A BatchNorm module whose parameters are frozen is therefore invisible to both code paths — the validator does not reject it, and no per-sample-gradient machinery is attached to it. After `make_private_with_epsilon` returns, the BatchNorm parameters are unfrozen so that the plain optimizer can update them locally. The freeze is thus a narrow, surgical maneuver: it lasts only for the duration of the privacy-engine setup call, and its sole purpose is to keep BatchNorm out of Opacus’s per-sample-gradient accounting while leaving it fully trainable during the actual training loop.

This is also why **two** optimizers are required rather than one. A single Opacus-wrapped optimizer covering all parameters would attempt to compute per-sample gradients for the BatchNorm parameters — the very thing that is ill-defined and that the validator forbids. By giving the DP-wrapped optimizer opt_A only the non-BN parameters and giving the plain optimizer opt_B the BN parameters, the construction routes each parameter group through exactly the update rule it needs: clipped-and-noised for the parameters that cross the boundary, plain for the parameters that stay home. The two optimizers do not interfere, because their parameter sets are disjoint and partition the model.

The result is a **two-optimizer training step**. Each client constructs two optimizers:

- opt_A — an AdamW [17] optimizer over the non-BN parameters $\theta^{\overline{BN}}$. This optimizer is the one wrapped by Opacus’s `PrivacyEngine`, so it performs DP-SGD per-sample gradient clipping and Gaussian noise addition on the non-BN gradients only.

- opt_B — a plain AdamW optimizer over the BN parameters θ^{BN} . This optimizer applies standard, un-noised gradient updates.

At each mini-batch, both optimizers are zeroed, a single forward and backward pass is run, and then both step: opt_A applies the DP-clipped, noised update to the non-BN parameters, and opt_B applies the plain update to the BN parameters. The two share the same backward pass; what differs is whether the resulting gradients pass through Opacus’s clipping-and-noise machinery before the step. Because opt_A applies DP-SGD with **per-sample** gradient clipping to the parameters that cross the boundary, the guarantee it produces is a **record-level** (ϵ, δ) -DP guarantee — this is the precise sense in which the construction differs from, and the source paper positions as stronger than, the party-level Laplace baseline of ADCOL [1].

On the server side, nothing changes from FedBN: the aggregation rule is Equation (4.1) — average the non-BN parameters across clients weighted by sample count, and do not touch the BN state. The server never sees any client’s BN state, so no DP composition is required for it. Differential privacy of an aggregate of DP-protected updates follows by post-processing: once each client’s transmitted $\theta^{\overline{BN}}$ update is DP, any function of those updates — including the size-weighted average the server computes — remains DP.

4.5.1 The three differentially private modes

The construction above is best understood next to the two alternatives it is compared against in the experiments. At finite ϵ , the BatchNorm-heavy CNN1D of Section 4.2 can be combined with DP in exactly three ways, and the implementation realizes all three so that the comparison is like-for-like:

1. **DP-FedAvg (raw BN)** — the naive composition: attach the privacy engine to the unmodified, BatchNorm-containing model and aggregate by FedAvg. This is what a practitioner would try first. It does not run: Opacus’s ModuleValidator rejects the model at setup time. In the implementation, the resulting exception is caught and re-raised as a typed “DP mode unsupported” error so that the experiment harness records the outcome as a first-class empirical finding (a REFUSED cell in the result tables of Chapter 6) rather than crashing. The refusal is the empirical fact that motivates any non-naive composition.

2. **DP-FedAvg+GroupNorm** — the standard Opacus workaround: replace every BatchNorm layer with GroupNorm up front via `ModuleValidator.fix` (Opacus’s default substitution replaces a k -channel BatchNorm with `GroupNorm(gcd(32, k), k)`), then attach the privacy engine to the now-DP-compatible model and aggregate by FedAvg. This runs, and it is the conventional baseline, but — as Chapter 2 explained — it has no per-client BatchNorm state, so it forfeits FedBN’s personalization mechanism. When BatchNorm is replaced, both the server’s reference model and every client are constructed with GroupNorm so that the parameter key spaces stay aligned for aggregation.
3. **DP-FedBN** — the construction of this chapter: freeze BatchNorm, attach the privacy engine to the non-BN parameters only, then run the two-optimizer step with FedBN aggregation. This is the only one of the three that obtains a record-level DP guarantee while keeping BatchNorm — and therefore FedBN’s mechanism — alive.

For the no-DP reference point ($\epsilon = \infty$) the privacy engine is omitted entirely and training reduces to plain FedBN, FedAvg, or FedAvg+GroupNorm; the GroupNorm topology is still applied in that mode because it is a property of the architecture choice, not of the privacy budget. These $\epsilon = \infty$ rows isolate the effect of *plumbing in* the DP infrastructure from the effect of the DP *noise* itself, a distinction Chapter 6 uses repeatedly.

4.6 Handling batch-normalization statistics under DP and FL

It is worth being precise about *which* parts of the BatchNorm state receive *which* treatment, because the mechanism by which DP affects FedBN, analyzed in Chapter 6, depends on it.

- The **running statistics** (μ, σ^2) are not optimized parameters at all. They are exponential moving averages updated by the forward pass (Section 2.4). They are updated normally during local training and they receive **no DP noise directly**, because they are never the target of either optimizer.
- The **affine parameters** (γ, β) are optimized, but by the plain optimizer opt_B , not by the DP-wrapped opt_A . They therefore receive standard gradient updates and are **not noised either**.
- The **non-BN parameters** $\theta^{\overline{BN}}$ are the only objects to which DP noise is applied, via opt_A .

The consequence — important for interpreting the results — is that DP noise enters the BatchNorm behavior only *indirectly*. The noised non-BN updates change the convolutional features, which change the activation statistics that feed the

BatchNorm running buffers; so the BN statistics drift under DP even though no noise is ever added to them directly. There is, moreover, no averaging across clients to smooth this drift, because FedBN deliberately keeps the BN state local. Chapter 6 shows that this is the mechanism behind DP-FedBN’s partition-dependent behavior.

4.7 Privacy accounting

The privacy guarantee is established and tracked per client. When the privacy engine is attached, Opacus calibrates the noise multiplier σ to the target (ϵ, δ) over the full course of that client’s training — that is, over $R \times E$ local epochs — using its RDP accountant and the sampling rate implied by the batch size and local dataset size (Section 2.5). The guarantee is therefore **end-to-end** over all rounds of the client’s participation, not per round. During training, the accountant is queried each round to report the cumulative spent ϵ , which rises toward the target as rounds accumulate; the achieved ϵ at the end of training matches the target to within 0.6% across all finite- ϵ runs reported in this thesis. Each client maintains its own accountant over its own data; because the server only ever combines DP-protected updates, the per-client (ϵ, δ) guarantees are not weakened by aggregation.

4.8 Algorithm and implementation pitfalls

Algorithm 1 states the full client-side procedure for one round, including the freeze-then-unfreeze setup.

Algorithm 1 — DP-FedBN: client-side setup and one round

Require: initial model f_θ with BN layers; train loader L_k ; target privacy budget (ϵ, δ) ; gradient clipping bound C .

Round setup (after global broadcast) 1. Receive $\theta_t^{\overline{BN}}$; load into the local model. 2. **for** each BN module m **do** $m.\text{weight.requires_grad} \leftarrow \text{False}$; $m.\text{bias.requires_grad} \leftarrow \text{False}$ 3. Build $opt_A \leftarrow \text{AdamW}(\theta^{\overline{BN}})$. 4. $(f_\theta, opt_A, L_k) \leftarrow \text{PrivacyEngine.make_private_with_epsilon}(\dots)$ 5. **for** each BN module m **do** $m.\text{weight.requires_grad} \leftarrow \text{True}$; $m.\text{bias.requires_grad} \leftarrow \text{True}$ 6. Build $opt_B \leftarrow \text{AdamW}(\theta^{BN})$ (plain, no DP).

Local training (each minibatch B) 7. $opt_A.\text{zero_grad}()$; $opt_B.\text{zero_grad}()$ 8. $\hat{y} \leftarrow f_\theta(B.x)$; $\ell \leftarrow \text{loss}(\hat{y}, B.y)$ 9. $\ell.\text{backward}()$ 10. $opt_A.\text{step}()$ $\triangleright$ DP-clipped + noised, on non-BN 11. $opt_B.\text{step}()$ $\triangleright$ plain, on BN

Round end 12. Send $\theta^{\overline{BN}}$ to the server. 13. BN parameters and running statistics remain local.

Two implementation pitfalls bit us during development and would bite a re-implementer the same way; both are non-obvious and are documented here because they are part of what makes the construction usable in practice.

State-dict key prefixing under GradSampleModule. When Opacus wraps a model, the underlying GradSampleModule prepends `_module.` to every parameter and buffer key in the model’s `state_dict()`. Any code path that snapshots or restores client model state — and FedAvg/FedBN parameter exchange is exactly such a path — must use the same unwrapping convention consistently, or the keys silently fail to match across the wrapped and unwrapped views. We standardized on a single accessor that returns the inner, unwrapped module and route every `state_dict` interaction through it, so that the aggregated parameter key space is always the plain one (`conv1.weight`, not `_module.conv1.weight`).

ExpandedWeights incompatibility. Opacus offers an alternative per-sample-gradient backend, ExpandedWeights (`grad_sample_mode='ew'`), which is faster than the default hooks-based backend. We found it incompatible with both the BatchMemoryManager’s logical-batch chunking and the `optimizer.zero_grad(set_to_none=True)` pattern: combining them crashes on the second optimizer step because ExpandedWeights accumulates gradients across calls without clearing them. We reverted to the default hooks-based backend, which — paired with a physical batch size of 96 and four data-loader workers — ran about six times faster than our initial configuration. These choices are recorded so that the privacy guarantee is reproducible exactly, rather than only in spirit.

5. EXPERIMENTAL SETUP

This chapter specifies the experimental protocol in full: the two ECG datasets and their preprocessing, the six non-IID partition schemes used to simulate heterogeneity, the model and training configuration, the differential-privacy parameters, the two evaluation regimes, and the implementation and reproducibility details. Every value reported here is taken from the source paper and the released code; nothing is reconstructed or assumed.

5.1 Datasets and preprocessing

Two datasets are used, chosen so that the central empirical claim can be tested on two deliberately different tasks: beat-level multi-class classification at one sampling rate, and record-level binary classification at another.

5.1.1 MIT-BIH Arrhythmia Database

The MIT-BIH Arrhythmia Database [18] is used for beat-level five-class classification under the ANSI/AAMI EC57 standard [19], [20], [21]. The database comprises 48 half-hour two-channel ambulatory ECG recordings sampled at 360 Hz. Each beat is read on channel 0 (column 0 of the WFDB signal). R-peak locations are taken from the standard .atr annotations, and each beat is windowed as ± 125 samples around its R-peak — 250 samples in total, approximately 694 ms at 360 Hz. Beats whose R-peak falls within 125 samples of either signal edge are dropped because their window would not fit. Each beat window is z-score-normalized.

Beat labels follow the AAMI five-class mapping, which groups the raw annotation symbols as

$$N = \{N, L, R, e, j\}, S = \{A, a, J, S\}, V = \{V, E\}, F = \{F\}, Q = \{/, f, Q\}.$$

Non-beat annotation symbols (rhythm markers, signal-quality markers, and the like) are dropped rather than forced into the Q class, and the four paced-record subjects (records 102, 104, 107, 217) are retained with their paced (/) beats labelled Q, following EC57 and de Chazal et al. [20]. The final dataset contains **109,453 beats across 48 records**. Its class distribution is severely imbalanced — $N \approx 82.8\%$ (90,595 beats), $Q \approx 7.3\%$ (8,040), $V \approx 6.6\%$ (7,235), $S \approx 2.5\%$ (2,781), and $F \approx 0.7\%$ (802) — which is the reason macro F1, not accuracy, is the primary metric (Section 5.5).

5.1.2 PTB-XL

PTB-XL [22] is used for record-level binary classification (Normal versus Abnormal). The 100 Hz records100/ variant is used. For each record, the diagnostic super-class is taken to be the SCP code with the highest annotated confidence whose diagnostic_class field is non-null in scp_statements.csv; records for which no such code exists are dropped. The binary task is then NORM versus the union of the four abnormal super-classes $\{MI, STTC, CD, HYP\}$. Each record uses **Lead II only** (column 1 of the 12-lead matrix), z-score-normalized, 1000 samples — ten seconds at 100 Hz. The final dataset contains **21,388 records**, of which 9,246 are NORM and 12,142 Abnormal, a roughly balanced split. The canonical PTB-XL stratified folds are preserved: folds 1–9 enter the client partitions and fold 10 serves as the central test set.

5.1.3 Why a binary task on PTB-XL

The choice of a binary PTB-XL task is deliberate and is documented here because it is a design decision rather than an arbitrary simplification. We initially attempted the PTB-XL five-super-class classification on Lead II. Three successive training regimes — a 44k-parameter model, then a 437k-parameter model with data augmentation and stronger regularization, and a cosine learning-rate schedule — all plateaued at a test macro F1 of approximately 0.50, with the hypertrophy (HYP) class stuck near 0.22. This was a representational bottleneck of single-lead input, not a capacity ceiling: the information needed to separate all five super-classes is not reliably present in Lead II alone. We therefore reformulated PTB-XL as a binary task (Normal versus Abnormal), which is well-defined on Lead II and reaches a balanced central macro F1 of approximately 0.81. Full multi-label PTB-XL classification using all twelve leads is a natural extension and is listed as future work in Chapter 7.

5.2 Non-IID partitioning

Because both datasets are pooled corpora rather than natively federated collections, statistical heterogeneity is *simulated* by partitioning each dataset across the clients with controlled non-IID schemes. Six partition strategies are used on MIT-BIH:

- **IID** — a stratified random split: each class is divided into roughly equal chunks across clients, so every client sees the same class proportions.
- **Quantity skew** ($\beta = 0.5$) — client sizes follow a power law drawn from a Dirichlet distribution, so the smallest client holds about half the data of the median client, while class proportions remain roughly IID.
- **Dirichlet** $\alpha \in \{1.0, 0.5, 0.1\}$ — each class’s allocation across clients is drawn from a Dirichlet prior; smaller α yields greater label heterogeneity. $\alpha = 1.0$ is mild, $\alpha = 0.1$ is severe.
- **Label skew** ($C = 2$) — each client owns exactly two of the five classes; this is the most extreme label heterogeneity available, with each class guaranteed to appear in at least one client.

The Dirichlet scheme is the standard mechanism for synthesizing label heterogeneity [28], [29] and is worth stating concretely. For each class c , a

proportion vector $(p_{c,1}, \dots, p_{c,K})$ is drawn from a symmetric Dirichlet distribution $Dir(\alpha, \dots, \alpha)$, and that class’s samples are distributed across the K clients in those proportions. A small α produces proportion vectors concentrated on a few clients — strong skew — while a large α pushes the proportions toward uniformity and hence toward IID. Quantity skew applies the same Dirichlet draw to client *sizes* rather than per-class shares.

A consequence of this design that must be stated plainly is that the heterogeneity here is **simulated on a pooled dataset** rather than measured across genuinely separate institutions. This is a deliberate trade-off — it gives precise, reproducible control over the type and severity of non-IID-ness, at the cost of not capturing real cross-institutional covariate shift (different devices, populations, and acquisition protocols). The implication for the results is discussed in the limitations (Chapter 6).

For the binary PTB-XL task only three of these are well-defined: IID, Dirichlet $\alpha = 0.1$, and label skew $C = 1$. The latter is the binary analogue of MIT-BIH’s $C = 2$ label skew, but it produces degenerate single-class clients (each client holds exactly one of the two classes); the consequences of this degeneracy are documented in Chapter 6 and that partition is excluded from the substantive DP-FedBN claims. All partitions are seeded deterministically — each client’s shard depends on the global seed plus its client index — so a partition is reproducible exactly.

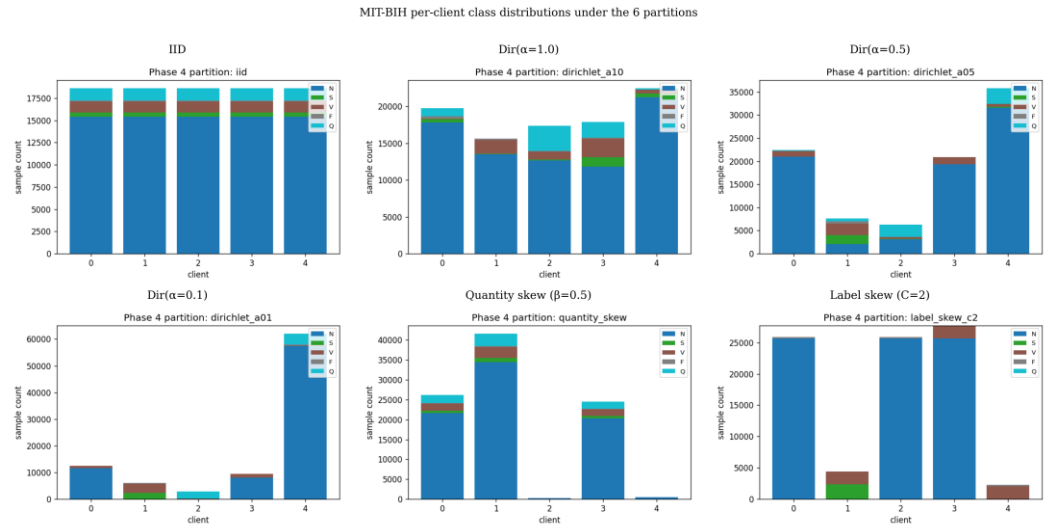


Figure 5.1. Six-panel stacked-bar visualization of per-client class distributions for all six MIT-BIH partitions across the five clients.

Figure 5.1 visualizes the six MIT-BIH partitions as per-client class-distribution stacked bars across the five clients. The progression from IID (every client a near-replica of the global distribution) through the Dirichlet schemes (increasingly lopsided class shares) to label skew $C = 2$ (each client a tall bar of just two colours) makes the controlled severity of the heterogeneity visible at a glance.

5.3 Model and training configuration

The model is the BatchNorm-heavy 1-D CNN described in Section 4.2 (approximately 437k parameters), used unchanged across all datasets, partitions, and privacy budgets. All federated experiments use $K = 5$ clients.

Each federated round runs $E = 5$ local epochs of mini-batch training per client at a batch size of 96. The total number of rounds is $R = 50$ for the non-DP experiments and $R = 30$ for the DP experiments. The optimizer is AdamW [17] with learning rate 1×10^{-3} and weight decay 5×10^{-4} ; the server aggregates after each round. The cross-entropy loss is weighted by inverse class frequency, computed on the pooled training shards so that rare classes are not drowned out — the same weighting scheme the centralized baselines use. A standard 80/20 stratified train/validation split is taken within each client’s shard; the validation slice is never used for gradient updates and serves both for convergence monitoring and, at the end of training, as the per-client local test set (Section 5.5).

The four non-DP aggregation strategies are FedAvg, FedProx, FedBN, and FedPerf. FedProx uses a proximal coefficient $\mu = 0.01$. FedBN excludes the BatchNorm parameters and buffers from aggregation per Equation (4.1). FedPerf is a personalization-aware baseline that weights client updates according to their local validation macro F1 (so that better-performing clients contribute more); it is included only to contextualize FedBN and is not claimed as a contribution.

Centralized baseline. To establish an upper bound that any federated configuration could at best approach, a single-machine model is trained on the full pooled training data of each dataset. The centralized runs use the same CNN1D architecture and AdamW optimizer, trained for up to 50 epochs with early stopping on validation macro F1 (patience 15 for MIT-BIH, 10 for PTB-XL). For the harder PTB-XL task, light label-preserving signal augmentation is applied during training — a random temporal crop, additive Gaussian noise in normalized units, and a slow sinusoidal baseline wander, each applied stochastically — which is the same

augmentation referenced in the five-super-class attempt of Section 5.1.3. These centralized numbers are reported in Chapter 6 as the pooled ceiling; they are not themselves a federated result.

5.4 Differential-privacy configuration

The differential-privacy experiments sweep three privacy budgets, $\varepsilon \in \{1, 3, \infty\}$, at a fixed $\delta = 10^{-5}$. The budget $\varepsilon = \infty$ denotes no DP noise and is used to separate the effect of plumbing in the DP infrastructure from the effect of the noise itself. For the finite budgets, the noise multiplier σ is chosen per client by Opacus’s `make_private_with_epsilon` accountant at batch size 96, so that each client’s training meets its target (ε, δ) guarantee end-to-end. The per-sample gradient clipping bound is $C = 1.0$. Opacus 1.5 is used with its default hooks-based per-sample-gradient backend (not the ExpandedWeights backend, for the reason given in Section 4.8), and the BatchMemoryManager bounds the physical batch size during per-sample gradient computation.

Three DP methods are compared, exactly the three of Section 4.5.1:

Method	BatchNorm handling	Expected behaviour
DP-FedAvg (raw BN)	unmodified BatchNorm	refused by Opacus’s ModuleValidator at finite ε
DP-FedAvg+GroupNorm	BatchNorm replaced by GroupNorm	runs; the standard recommended workaround
DP-FedBN	BatchNorm frozen from DP, kept local	runs; the construction of this thesis

The DP sweeps are run on the two most challenging MIT-BIH partitions (label skew $C = 2$ and Dirichlet $\alpha = 0.1$) and on the meaningful PTB-XL partition (Dirichlet $\alpha = 0.1$), spanning the three methods and three budgets. Because the privacy budget is a *target* that the accountant calibrates the noise to meet, the actually *achieved* ε is recorded at the end of each finite- ε run by querying the accountant; across all finite- ε runs the achieved budget lands within 0.6% of the

target (for example, a target of 3 is achieved at approximately 2.99), confirming that the privacy guarantee is calibrated as intended rather than merely nominal.

5.5 Evaluation regimes and metrics

A central methodological commitment of this thesis is that **every run is evaluated under two regimes**, because the choice between them is itself one of the findings (Chapter 6):

- **Central (macro) F1** — the final model that is most representative of the federation is evaluated on a pooled, held-out central test set, and macro F1 is reported. For FedAvg, FedProx, and FedPerf, the representative model is the single global model; for FedBN, which has no single global model, both the first client’s personalized model and the per-client average are evaluated. The central test set is a 15% stratified holdout for MIT-BIH and the canonical fold 10 for PTB-XL.
- **Local (macro) F1** — each client’s final personalized model is evaluated on its own held-out validation slice, drawn from that client’s own (non-IID) data distribution; the mean and standard deviation across the five clients are reported.

The distinction is exactly the pooled-versus-per-client distinction foreshadowed in Section 2.3: central F1 measures performance on the global data mixture, local F1 measures per-client performance on each client’s own distribution. The primary metric in both regimes is **macro F1**, preferred over accuracy because of the severe class imbalance on MIT-BIH (Section 5.1.1); macro-averaged one-versus-rest AUC is reported as a secondary metric. For the central evaluation, the reported headline number for a run is the best macro F1 achieved across rounds; local F1 is measured at the end of training. Because FedBN has no single global model, its central evaluation is computed in two ways — the first client’s personalized model evaluated on the central test set, and the average of all clients’ personalized models each evaluated on it — and both are logged; the qualitative finding (Chapter 6) does not depend on which is chosen.

One caveat on the MIT-BIH central test set should be stated for completeness. It is a 15% stratified holdout drawn at the *beat* level, so beats from a given patient may appear in both the client pool and the central test set; this is the intra-patient evaluation setting discussed in Section 3.4, and it is less strict than a patient-disjoint

inter-patient split. The PTB-XL central test set, being the canonical fold 10, respects the dataset’s patient-aware fold assignment. This caveat is noted again among the limitations.

5.6 Implementation, reproducibility, and hardware

All federated experiments are run on a **custom, sequential, in-process simulator** rather than a full federated framework such as Flower. The choice is pragmatic: a minimal in-process simulator avoids the inter-process-communication overhead that would otherwise be incurred across the 100-plus runs in the experimental matrix. The four sweep scripts are resumable — a run is skipped if its results file already exists — which made the multi-day experimental campaign robust to interruption.

Reproducibility is pinned to a single random seed, 42, used everywhere. `torch.backends.cudnn.deterministic` is set to `True`, all random-number generators are seeded, and the git commit hash is logged with every run. Following the source paper, `cudnn.benchmark` is left `True` for convolution-kernel autotuning; this makes kernel selection non-deterministic *across* machines but stable on a given GPU and PyTorch combination, so results are reproducible on the same hardware. The software stack is PyTorch 2.4 and Opacus 1.5. The hardware was an NVIDIA GTX 1660 Super (6 GB) for the initial phases and an NVIDIA RTX 4070 from the PTB-XL cross-task replication onward.

One disclosure is important enough to state here as well as in the limitations. **All reported numbers are single-seed realizations** (seed 42). Quantifying the variance across seeds is the single most important deferred experiment, and it is discussed as a limitation in Chapter 7. The released code, configurations, and per-run logs make every number in this thesis reproducible on matching hardware.

6. RESULTS AND DISCUSSION

This chapter reports the experimental results in five threads — the centralized baselines that pin the upper bound, the non-IID federated benchmark on MIT-BIH, the eval-regime flip, the differential-privacy privacy–utility frontier on MIT-BIH, and the cross-task replication on PTB-XL — and then interprets them: why DP-FedBN’s advantage is partition-dependent, what the findings imply for practitioners, and why the choice of evaluation regime is itself a result. The chapter

closes with the limitations. Every number reported is a single-seed (seed 42) realization, a point whose consequences are addressed in Section 6.7.

6.1 Centralized baselines

A single-machine model trained on the full pooled training data establishes the ceiling that any federated configuration could at best approach. On MIT-BIH five-class beat classification it reaches a test **macro F1 of 0.913**, and on PTB-XL binary record classification a test **macro F1 of 0.813**. These two numbers are the reference points for everything that follows: they pin the “IID-but-federated” performance that the federated runs are measured against, and they confirm that the binary PTB-XL reformulation (Section 5.1.3) reaches a healthy, balanced operating point where the five-super-class attempt had stalled near 0.50.

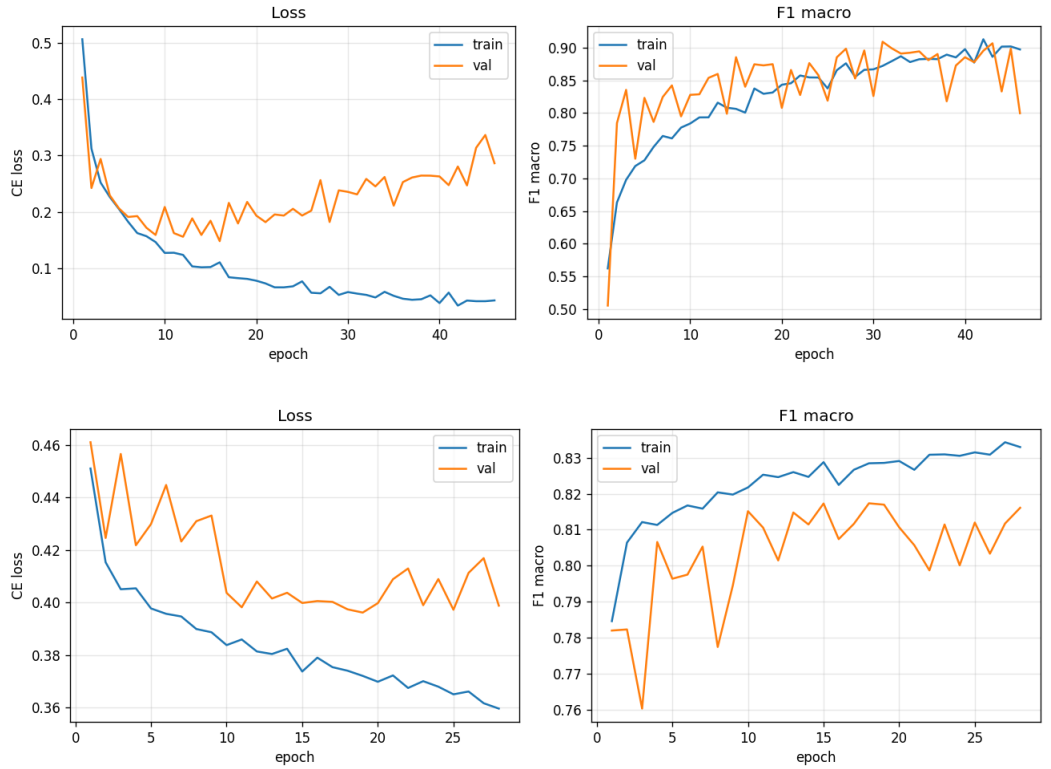


Figure 6.1. Per-epoch training/validation loss and macro-F1 curves for the two centralized baselines.

The per-epoch training and validation curves for the two centralized models (Figure 6.1) show stable convergence with early stopping on validation macro F1, and serve as a sanity check that the shared architecture is well-matched to both tasks before any federation or privacy machinery is introduced.

6.2 Non-IID federated benchmark on MIT-BIH

Table 6.1 reports the best central macro F1 for each combination of partition and aggregation on MIT-BIH, across the full 6×4 grid of six partitions and four aggregation strategies.

Table 6.1 — MIT-BIH five-class non-IID benchmark: best central macro F1 across 50 rounds (single seed). Higher is better.

Partition	FedAvg	FedProx	FedBN	FedPerf
IID	0.926	0.860	0.920	0.926
Quantity skew ($\beta = 0.5$)	0.922	0.884	0.912	0.913
Dirichlet $\alpha = 1.0$	0.937	0.889	0.911	0.928
Dirichlet $\alpha = 0.5$	0.926	0.903	0.850	0.928
Dirichlet $\alpha = 0.1$	0.745	0.735	0.554	0.745
Label skew ($\mathcal{C} = 2$)	0.527	0.386	0.219	0.530

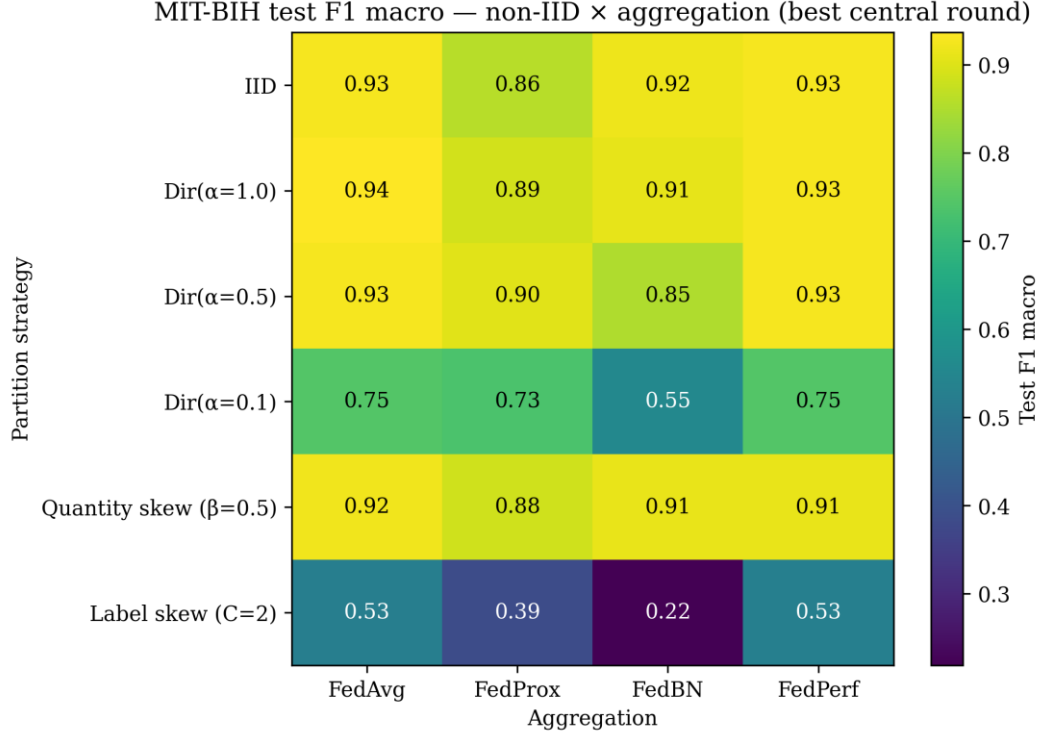


Figure 6.2. Heatmap of best-central-round macro F1 across the partition $\times$ aggregation grid; FedBN’s label-skew cell (0.22) is the visual outlier.

Three patterns are visible, and they are reinforced by the heatmap view of the same grid in Figure 6.2.

First, in the IID-and-mild-skew partitions — IID, quantity skew, and Dirichlet $\alpha \geq 0.5$ — every aggregation sits close to the centralized 0.913 ceiling. Federation costs essentially nothing in this benign regime; the choice of aggregation rule barely matters.

Second, severe label heterogeneity — Dirichlet $\alpha = 0.1$ and label skew $C = 2$ — collapses every aggregation, but **FedBN collapses the most**. On label skew $C = 2$, FedBN reaches a central macro F1 of only 0.219, while FedAvg, FedProx, and FedPerf reach 0.527, 0.386, and 0.530 respectively. This is a striking result, because it contradicts the original FedBN paper [9], which reported FedBN *outperforming* FedAvg under non-IID data. The apparent contradiction is the entry point to the central finding of this thesis and is resolved in Section 6.3.

Third, FedProx at its un-tuned proximal coefficient $\mu = 0.01$ is consistently worse than FedAvg across every partition. This is a known sensitivity of the proximal term to its hyperparameter rather than a deficiency of the method, and tuning μ is left to future work.

6.3 The eval-regime flip

The collapse of FedBN’s central F1 on label-heterogeneous data was surprising enough to warrant a careful audit of the implementation before any conclusion was drawn (described at the end of this section). That audit’s diagnostic check measured, for the FedBN runs, the per-client *local* macro F1 alongside the central macro F1 — and the result is the empirical core of this thesis. Table 6.2 reports both regimes on the most extreme partition, label skew $C = 2$.

Table 6.2 — MIT-BIH label skew $C = 2$: central macro F1 (pooled holdout) versus per-client local macro F1 (mean $\pm$ std across 5 clients). FedBN ranks worst centrally and best locally.

Aggregation	Central F1	Local F1 (mean $\pm$ std)	Δ (local – central)
FedAvg	0.527	0.823 ± 0.178	+0.297
FedProx	0.386	0.488 ± 0.380	+0.101
FedBN	0.224	0.942 ± 0.053	+0.718
FedPerf	0.530	0.654 ± 0.255	+0.124

The numbers tell a single, sharp story. FedBN’s central F1 is 0.224 — the lowest of the four aggregations — while its local F1 is 0.942 — the highest of the four. **The same model, with the same training and the same federation, ranks last under one evaluation regime and first under the other.** The central-versus-local gap reaches +0.72, the largest in the table by a wide margin. This is the *eval-regime flip*.

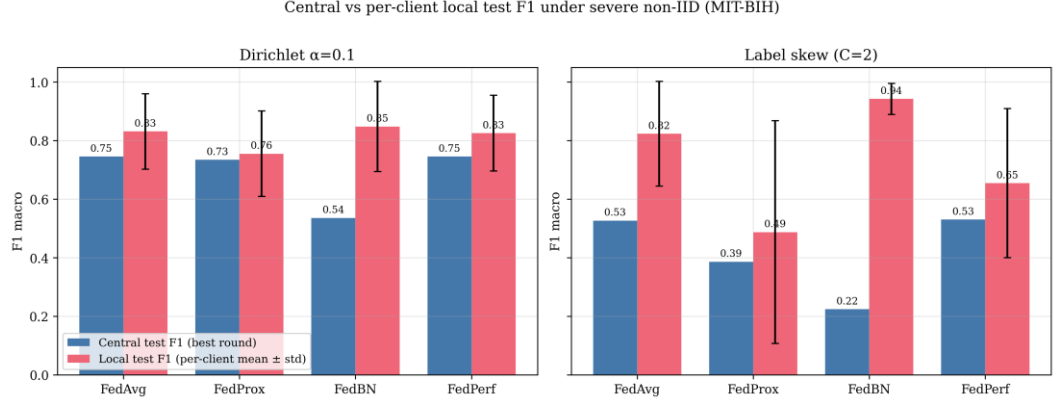


Figure 6.3. Grouped bars of central vs per-client local macro F1 for the four aggregations, on Dirichlet $\alpha=0.1$ and label skew $C=2$; FedBN is lowest on central, highest on local.

Figure 6.3 visualizes the flip across both severe partitions. On label skew $C = 2$, FedBN’s central bar is the shortest while its local bar is the tallest; the pattern is the same, with smaller magnitude, on Dirichlet $\alpha = 0.1$.

Mechanism. The flip has a clean explanation rooted in what FedBN keeps local: the BatchNorm running statistics. Each client’s BN statistics are calibrated to *that client’s* local label distribution. Under label skew $C = 2$, a client owns only two of the five classes, so its BN statistics are tuned for the activation distribution of those two classes. When that client’s full model — convolutional features shared across the federation, but BN specialized to two classes — is asked to classify the *central* test set, which contains all five classes in MIT-BIH’s natural proportions, its BN normalization collapses on the three classes the client never trained on. Pooled-central evaluation therefore punishes FedBN’s specialization. Per-client local evaluation, where each client is only ever asked about its own distribution, rewards exactly that specialization, which is why FedBN’s local F1 is the highest of the four.

The same pattern holds on the milder Dirichlet $\alpha = 0.1$ partition with smaller magnitude: FedBN’s central F1 is 0.536 versus a local F1 of 0.848 ± 0.154 , a +0.31 flip, while FedAvg shows a central 0.745 versus local 0.831 ± 0.128 , a +0.09 flip. The sign and the ordering are preserved; only the magnitude shrinks as the heterogeneity softens.

Implementation audit. Because this result contradicts the published FedBN finding, the collapse was subjected to a multi-check audit before being accepted: verification of BatchNorm-key discovery, of the server-side BatchNorm exclusion, of post-round BatchNorm divergence across clients, of parameter-set integrity, of

IID parity (FedBN matching FedAvg when the data is IID), and a synthetic feature-shift sanity test that reproduces the regime FedBN was originally designed for. In that synthetic feature-shift setup, FedBN correctly recovers a **+0.45 central-F1 advantage over FedAvg**, exactly as the original paper would predict — confirming that the implementation is correct and that FedBN does help in the feature-shift regime it was built for. The central-F1 collapse on *label*-heterogeneous medical data is therefore a real property of the regime, not a bug: FedBN’s advantage and disadvantage are two faces of the same locality mechanism, and which one is observed depends on whether evaluation is pooled-central or per-client local.

6.4 Differential-privacy privacy–utility frontier on MIT-BIH

The three differentially private methods — DP-FedAvg (raw BatchNorm), DP-FedAvg+GroupNorm, and DP-FedBN — were run across $\epsilon \in \{1, 3, \infty\}$ on the two most challenging partitions, label skew $C = 2$ and Dirichlet $\alpha = 0.1$. Table 6.3 reports central and local F1 for every combination; the achieved ϵ values fall within 0.6% of target on all finite- ϵ runs.

Table 6.3 — MIT-BIH DP privacy–utility benchmark (single seed, $\delta = 10^{-5}$). Naive DP-FedAvg is refused by Opacus at finite ϵ .

Method	Partition	ϵ	Central F1	Local F1 (mean $\pm$ std)
DP-FedAvg (raw BN)	Label skew $C = 2$	∞	0.517	0.829 ± 0.192
DP-FedAvg (raw BN)	Label skew $C = 2$	3	REFUSED	—
DP-FedAvg (raw BN)	Label skew $C = 2$	1	REFUSED	—
DP-FedAvg+GroupNorm	Label skew $C = 2$	∞	0.261	0.463 ± 0.368
DP-FedAvg+GroupNorm	Label skew $C = 2$	3	0.191	0.295 ± 0.241

Method	Partition	ε	Central F1	Local F1 (mean $\pm$ std)
DP-FedAvg+GroupNorm	Label skew $C = 2$	1	0.181	0.295 ± 0.241
DP-FedBN	Label skew $C = 2$	∞	0.221	0.937 ± 0.043
DP-FedBN	Label skew $C = 2$	3	0.181	0.467 ± 0.264
DP-FedBN	Label skew $C = 2$	1	0.217	0.344 ± 0.194
DP-FedAvg (raw BN)	Dirichlet $\alpha = 0.1$	∞	0.738	0.816 ± 0.130
DP-FedAvg (raw BN)	Dirichlet $\alpha = 0.1$	3	REFUSED	—
DP-FedAvg (raw BN)	Dirichlet $\alpha = 0.1$	1	REFUSED	—
DP-FedAvg+GroupNorm	Dirichlet $\alpha = 0.1$	∞	0.736	0.832 ± 0.118
DP-FedAvg+GroupNorm	Dirichlet $\alpha = 0.1$	3	0.382	0.376 ± 0.101
DP-FedAvg+GroupNorm	Dirichlet $\alpha = 0.1$	1	0.378	0.350 ± 0.144
DP-FedBN	Dirichlet $\alpha = 0.1$	∞	0.551	0.861 ± 0.130
DP-FedBN	Dirichlet $\alpha =$	3	0.189	0.368 ± 0.151

Method	Partition	ϵ	Central F1	Local F1 (mean $\pm$ std)
	0.1			
DP-FedBN	Dirichlet $\alpha = 0.1$	1	0.251	0.465 ± 0.308

DP-FedBN preserves local-eval advantage under DP on label-skew, mixed on Dirichlet $\alpha=0.1$

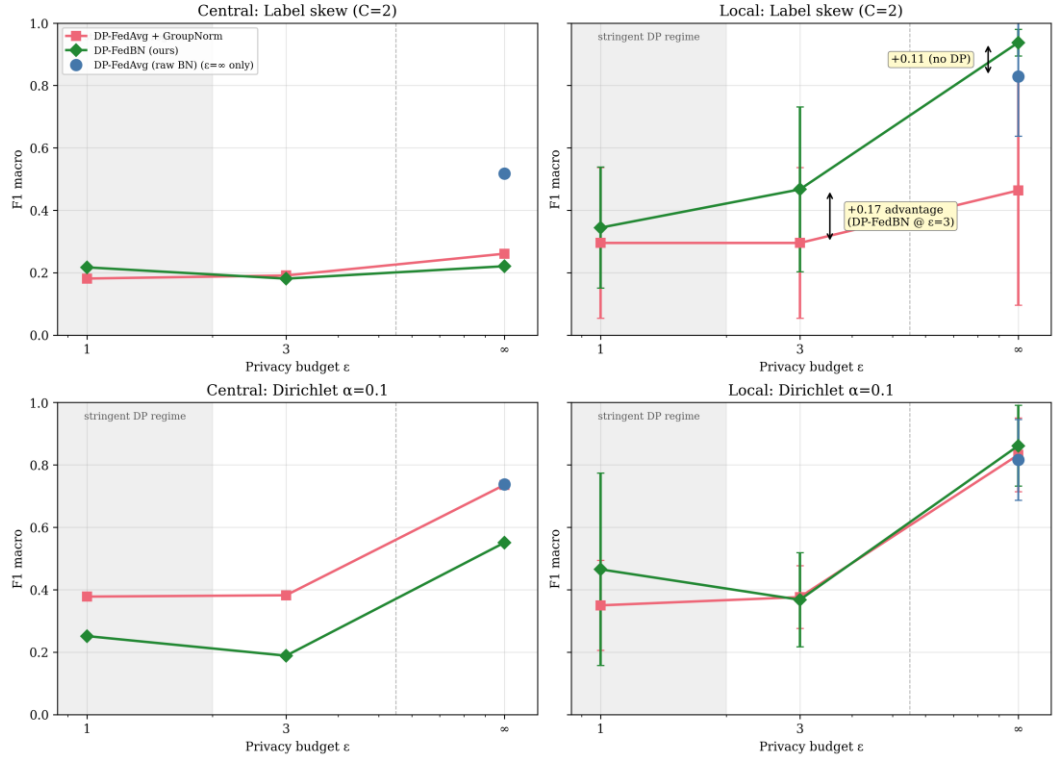


Figure 6.4. 2x2 grid (rows: label skew $C=2$, Dirichlet $\alpha=0.1$; columns: central, local) of macro F1 vs epsilon for the three DP methods.

Figure 6.4 plots this frontier as a 2×2 grid of central and local F1 against ϵ for the two partitions. Three observations follow.

First, **naive DP-FedAvg is empirically refused**. Four of the eighteen runs — raw BatchNorm at both finite budgets on both partitions — terminate at setup time because Opacus’s ModuleValidator will not accept a BatchNorm-containing model under unmodified DP-SGD. This is not an omission in the experimental matrix; it is the empirical fact that motivates a non-naive composition such as DP-FedBN or the GroupNorm fix, and it is recorded as a first-class REFUSED result rather than quietly dropped.

Second, **on label skew, DP-FedBN preserves a clear local edge.** At $\varepsilon = 3$, DP-FedBN’s local F1 is 0.467 versus 0.295 for DP-FedAvg+GroupNorm — a **+0.17 gap that survives differential privacy**. At the stringent $\varepsilon = 1$ the gap shrinks to +0.05 but remains positive, and at $\varepsilon = \infty$ (DP infrastructure but no noise) it is +0.47. FedBN’s specialization mechanism transfers under DP on this partition: even noised, a per-client BatchNorm tuned to two classes beats a globally-normalized GroupNorm on each client’s own data.

Third, **on Dirichlet, the local edge is mixed.** At $\varepsilon = 3$, DP-FedBN’s local F1 of 0.368 is effectively tied with DP-FedAvg+GroupNorm’s 0.376 — a -0.008 difference well within single-seed noise. At $\varepsilon = 1$, DP-FedBN wins again (0.465 versus 0.350, a +0.115 gap), and at $\varepsilon = \infty$ it wins by 0.029. The non-monotonic ordering across ε on this partition is a single-seed signature, and no strong claim is drawn from its precise shape; what is robust is that the large, decisive local edge seen on label skew is absent here.

6.5 Cross-task replication on PTB-XL

To test whether these findings are specific to MIT-BIH or hold as task-agnostic federated-learning phenomena, the non-DP and DP sweeps were replicated on PTB-XL binary record classification — a genuinely different task (ten-second records rather than 250-sample beats, 100 Hz rather than 360 Hz, two classes rather than five).

Non-DP: the eval-regime flip is confirmed. Table 6.4 reports the twelve non-DP runs across three partitions and four aggregations.

Table 6.4 — PTB-XL binary, non-DP cross-task replication (best central F1; local mean $\pm$ std).

Partition	Aggregation	Central F1	Local F1 (mean $\pm$ std)
IID	FedAvg	0.806	0.817 $\pm$ 0.008
IID	FedProx	0.811	0.800 $\pm$ 0.007
IID	FedBN	0.797	0.691 $\pm$ 0.131

Partition	Aggregation	Central F1	Local F1 (mean $\pm$ std)
IID	FedPerf	0.812	0.789 ± 0.008
Dirichlet $\alpha = 0.1$	FedAvg	0.804	0.464 ± 0.148
Dirichlet $\alpha = 0.1$	FedProx	0.817	0.661 ± 0.113
Dirichlet $\alpha = 0.1$	FedBN	0.688	0.588 ± 0.103
Dirichlet $\alpha = 0.1$	FedPerf	0.805	0.419 ± 0.234
Label skew $C = 1$ †	FedAvg	0.363	0.400 ± 0.490
Label skew $C = 1$ †	FedProx	0.389	0.431 ± 0.363
Label skew $C = 1$ †	FedBN	0.326	1.000 ± 0.000
Label skew $C = 1$ †	FedPerf	0.397	0.462 ± 0.301

† On a binary task, label skew $C = 1$ gives every client exactly one class; FedBN’s local F1 is then trivially 1.000 (each client predicts its single class perfectly on its own single-class slice). This is documented as a partition design flaw, not a finding, and is excluded from the DP-FedBN claims.

On Dirichlet $\alpha = 0.1$, FedBN’s central F1 of 0.688 is the lowest of the four aggregations, while its local F1 of 0.588 beats FedAvg’s local 0.464 by **+0.123** — a *larger* absolute margin than the corresponding comparison on MIT-BIH Dirichlet (+0.017). The eval-regime flip — FedBN below FedAvg on central, above on local — survives a complete change of task, dataset, sampling rate, and class count. Figure 6.5 shows the two datasets side by side, with FedBN ranking below FedAvg on central F1 and above on local F1 in both.

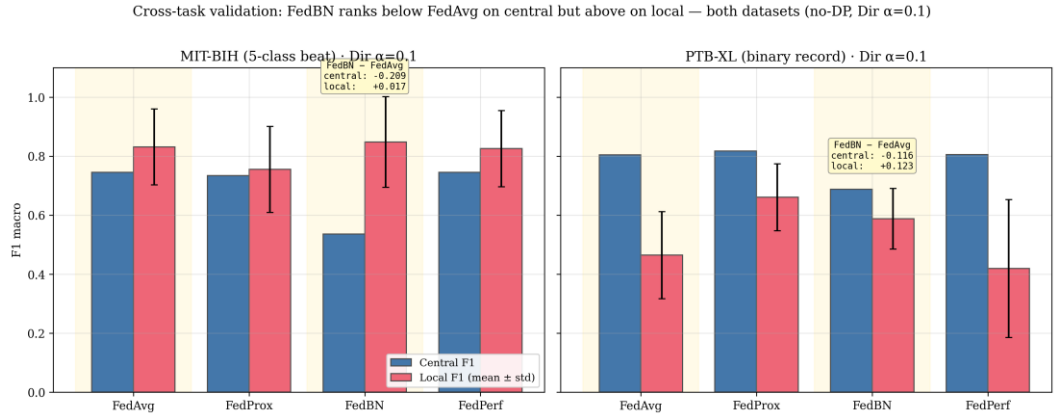


Figure 6.5. MIT-BIH and PTB-XL panels (Dirichlet $\alpha=0.1$, no DP), central vs local bars, annotated with the FedBN-minus-FedAvg gaps (central negative, local positive) on both datasets.

The label skew $C = 1$ rows illustrate why that partition is excluded from the substantive claims: with one class per client, FedBN’s local F1 is exactly 1.000 with zero variance at every budget, because each client’s model perfectly predicts its only class on its own single-class data. The result is invariant to method, partition, and DP noise; it is an artifact of the partition on a binary task, not an algorithmic property.

DP: DP-FedBN loses on PTB-XL Dirichlet. Table 6.5 reports the nine-cell DP sweep on the meaningful PTB-XL partition, Dirichlet $\alpha = 0.1$.

Table 6.5 — PTB-XL Dirichlet $\alpha = 0.1$ DP cross-task (best central F1; local mean $\pm$ std).

Method	ε	Central F1	Local F1 (mean $\pm$ std)
DP-FedAvg (raw BN)	∞	0.807	0.552 $\pm$ 0.162
DP-FedAvg (raw BN)	3	REFUSED	—
DP-FedAvg (raw BN)	1	REFUSED	—
DP-FedAvg+GroupNorm	∞	0.784	0.443 $\pm$ 0.142
DP-FedAvg+GroupNorm	3	0.774	0.669 $\pm$ 0.109

Method	ε	Central F1	Local F1 (mean $\pm$ std)
DP-FedAvg+GroupNorm	1	0.765	0.715 $\pm$ 0.148
DP-FedBN	∞	0.688	0.604 $\pm$ 0.125
DP-FedBN	3	0.400	0.435 $\pm$ 0.054
DP-FedBN	1	0.451	0.526 $\pm$ 0.146

At $\varepsilon = \infty$, DP-FedBN’s local F1 of 0.604 beats DP-FedAvg+GroupNorm’s 0.443 by +0.161, preserving the non-DP rank flip once the DP infrastructure is plumbed in but before any noise is added. At finite ε , however, the ordering **reverses**: DP-FedAvg+GroupNorm’s local F1 at $\varepsilon = 3$ is 0.669 versus DP-FedBN’s 0.435, a **−0.234** loss for DP-FedBN; at $\varepsilon = 1$ the ordering remains, 0.715 versus 0.526, a **−0.189** loss. On PTB-XL Dirichlet, DP noise does not merely erode DP-FedBN’s advantage — it inverts it.

Synthesis. Combining the MIT-BIH and PTB-XL DP results, **DP-FedBN’s local advantage is partition-dependent**, with a consistent sign across both datasets:

- Label skew with at least three classes per task (MIT-BIH $C = 2$): **+0.17 at $\varepsilon = 3$; DP-FedBN wins.**
- Dirichlet $\alpha = 0.1$: tied or losing under DP — MIT-BIH -0.008 and PTB-XL -0.234 at $\varepsilon = 3$, the same sign in a sharper magnitude; **DP-FedAvg+GroupNorm wins.**

6.6 Discussion

6.6.1 Why DP-FedBN’s edge is partition-dependent

FedBN’s no-DP local advantage comes from one design choice: keep each client’s BatchNorm running statistics local and never average them across clients. In a heterogeneous regime, the per-client BatchNorm is a *better* normalization for that client’s data than any globally-averaged BatchNorm could be, because it is

matched to the client’s own distribution. That is the whole source of FedBN’s local-eval strength.

Under differential privacy, this same locality becomes a liability whose severity depends on the partition. As established in Section 4.6, the BatchNorm running statistics and affine parameters receive no DP noise *directly* — the running buffers are forward-pass moving averages, and the affine parameters are updated by the plain optimizer. But the noised non-BatchNorm gradients change the convolutional features, and those features are exactly the activations whose statistics feed the BatchNorm running buffers. DP noise therefore disturbs each client’s BatchNorm behavior *indirectly*, and — crucially — there is **no averaging across clients to smooth that disturbance out**, because FedBN deliberately keeps the BatchNorm state local. Each client’s BatchNorm consequently drifts in directions driven more by upstream noise than by data structure, and it loses the very specialization that was its advantage. GroupNorm, by contrast, has no running statistics, and the DP noise that reaches its affine parameters *is* averaged across clients during aggregation, which damps it. This asymmetry — FedBN’s per-client normalization accumulates DP-driven drift without averaging, GroupNorm’s shared normalization averages it away — is the mechanism, and it is consistent with the sign of the result on both datasets.

Why, then, does label skew differ from Dirichlet? The answer is the *amount of specialization available to lose*. On label skew $C = 2$, each client owns only two of five classes, and the specialization that a per-client BatchNorm provides is so much better than a globally-averaged one that even a noised FedBN beats GroupNorm on local evaluation at $\epsilon = 3$. Specialization wins when the “specialization distance” between a client’s distribution and the global average is large. On Dirichlet $\alpha = 0.1$, each client has skewed-but-not-truncated class proportions — it still sees every class, just in unusual ratios — so the specialization distance is small, a globally-averaged BatchNorm is a reasonable normalizer for any client, and the DP noise that FedBN’s locality fails to average away costs more than the residual specialization gains. GroupNorm wins. The magnitude of DP-FedBN’s loss tracks how little specialization there was to protect.

6.6.2 A practitioner rubric for medical federated learning

These findings translate into a small, heterogeneity-typed decision rubric for medical FL practitioners:

1. **Label-truncated heterogeneity** (specialty hospitals, single-condition clinics, where each site sees only a few conditions): DP-FedBN preserves a clear local-F1 edge under DP, and the dual-optimizer overhead is justified.
2. **Proportional heterogeneity** (general hospitals with a mixed but complete case-mix): DP-FedAvg+GroupNorm is simpler and at least as good — often better.
3. **Local-deployment priority** (each site runs its own personalized model): FedBN without DP gives the strongest local F1; if DP is required, DP-FedBN is competitive on label-truncated data and competitive-or-worse on proportional data.
4. **Strict regulatory pooled-central requirements** (a single model evaluated on a pooled benchmark): FedAvg or FedPerf, not FedBN — the eval-regime flip means FedBN’s central F1 ranks worst even when its local F1 ranks best.

6.6.3 *The evaluation regime is a methodological variable*

The most transferable lesson is methodological. A reader who reports “central F1 on a held-out test pool” as the primary metric will conclude that FedBN underperforms. A reader evaluating the same models per-client may conclude that FedBN wins. Whether to report central or local F1 is therefore not a stylistic choice — **it changes the conclusion**. When the deployment scenario is itself ambiguous between a pooled, single-model setting and a per-site personalized-model setting, both regimes should be reported. Most federated-learning benchmarking papers, including the original FedBN paper [9], report only one regime; this thesis suggests that the regime is a missing variable whose omission can silently determine which method appears best.

6.7 Limitations

The findings should be read with the following limitations in mind, several of which are the natural starting points for future work (Chapter 7).

- **Single seed.** All reported numbers are single-seed realizations (seed 42). The non-monotonicities across ϵ on the Dirichlet partition are single-seed signatures, and multi-seed variance quantification is the single most important deferred experiment.

- **One architecture.** A fixed BatchNorm-heavy CNN1D (≈ 437 k parameters) is used throughout; whether the eval-regime flip and the partition-dependent DP behavior persist for ResNet-style or Transformer architectures is untested.
- **Simulated heterogeneity.** Non-IID partitions are constructed on pooled datasets rather than measured across genuinely separate institutions, so true cross-institutional covariate shift is not captured. Relatedly, the MIT-BIH central test set is a beat-level (intra-patient) stratified holdout (Section 5.5).
- **Five clients.** Production federated systems involve dozens to hundreds of participants; client-count effects are not characterized.
- **ECG-only validation.** The findings are demonstrated on two ECG tasks; their reach to other medical modalities is an open question.
- **Un-tuned FedProx.** FedProx uses $\mu = 0.01$ and consistently under-performs FedAvg, likely from insufficient hyperparameter tuning rather than a deficiency of the method.
- **Binary PTB-XL.** Multi-label PTB-XL over all five super-classes using the full twelve leads (with macro-AUC as the metric) is the natural extension and is left as future work.

7. CONCLUSION AND FUTURE WORK

7.1 Summary of contributions

This thesis presented and characterized **DP-FedBN**, a dual-optimizer composition of DP-SGD with FedBN that preserves per-client BatchNorm specialization while applying differential privacy only to the parameters that actually cross the federation boundary. The composition recognizes that, under FedBN, the BatchNorm parameters and running statistics never leave the client and therefore require no DP protection in the standard federated threat model; only the convolutional and linear parameters that are transmitted need DP. A freeze-then-unfreeze maneuver makes Opacus accept the BatchNorm-containing model, and two parallel optimizers — one wrapped by the privacy engine for the non-BatchNorm parameters, one plain for the BatchNorm parameters — route each parameter group through exactly the update rule it needs.

The contribution is stated in deliberately measured terms. The name “DP-FedBN” already appears in ADCOL (ICML 2023) [1], where it denotes a one-paragraph, party-level Laplace baseline on aggregated model updates; this work claims neither that concept nor a new aggregation rule nor a new privacy mechanism. To our knowledge, it is the **first record-level DP-SGD composition of FedBN’s BatchNorm-locality with explicit treatment of the Opacus engineering obstacle**, together with the first systematic medical empirical characterization of that composition across heterogeneity regimes and privacy budgets — two ECG tasks, six non-IID partition regimes, and three privacy budgets, totaling about forty configurations.

7.2 Principal findings

Three findings emerge from that characterization, each stated at the strength the evidence supports.

First, the eval-regime flip that FedBN induces under label heterogeneity is task-agnostic. FedBN underperforms in pooled-central evaluation while overperforming in per-client local evaluation, on both MIT-BIH five-class beats and PTB-XL binary records. On MIT-BIH label skew the central-versus-local gap reaches +0.72 macro F1; the same qualitative flip — FedBN below FedAvg on central, above on local — survives a complete change of task to PTB-XL. The broader lesson is that, in many realistic deployment scenarios, the choice between central and local evaluation matters more than the choice of aggregation method. It is a methodological variable that benchmarking studies, the original FedBN paper among them, have tended to leave implicit, and which can silently determine which method appears best.

Second, DP-FedBN’s local advantage under differential privacy is partition-dependent. It preserves a +0.17 local-F1 edge over DP-FedAvg+GroupNorm on MIT-BIH label skew at $\epsilon = 3$, where DP-FedBN wins; but it records −0.008 on MIT-BIH Dirichlet and −0.234 on PTB-XL Dirichlet at the same budget, where DP-FedAvg+GroupNorm wins. The mechanism is consistent in sign across both datasets: DP-driven activation noise propagates to each client’s per-client BatchNorm parameters without the cross-client averaging that smooths it for shared parameters, so FedBN’s locality — its strength without DP — becomes a liability under DP whose severity scales with how little class-truncation specialization there was to protect.

Third, naive DP-FedAvg on a BatchNorm-containing model is empirically refused by Opacus’s ModuleValidator at every finite ϵ . This is not a side observation but the concrete motivation for the dual-optimizer formulation: a practitioner cannot simply attach DP-SGD to a BatchNorm-heavy network, and the two principled responses are either to replace BatchNorm with GroupNorm or to keep it local as DP-FedBN does.

7.3 Practical guidance

For medical federated-learning practitioners, the findings reduce to a short, heterogeneity-typed rubric. Use DP-FedBN when sites are label-truncated — specialty clinics and single-condition cohorts — and local deployment of a per-site model is the goal; there it preserves a clear local-F1 edge under DP, and the dual-optimizer overhead is justified. Use DP-FedAvg+GroupNorm when sites have proportional, mixed-but-complete class heterogeneity, where it is simpler and at least as good. And report **both** central and local F1 whenever the deployment regime is itself ambiguous between a single pooled model and per-site personalized models, because the two regimes can disagree on which method is best.

7.4 Limitations

The principal limitations bound the scope of these claims and motivate the future work below. All numbers are single-seed (seed 42), so seed variance is unquantified and the non-monotonicities observed across ϵ should be read as single-seed signatures. A single fixed BatchNorm-heavy CNN1D architecture is used throughout. Heterogeneity is simulated by partitioning pooled datasets rather than measured across genuinely separate institutions, and the MIT-BIH central test set is an intra-patient stratified holdout. The federation has only five clients, where production systems have dozens to hundreds. FedProx is run at a single un-tuned proximal coefficient. The PTB-XL task is reduced to binary on a single lead. Validation is confined to ECG.

7.5 Future work

Several concrete directions follow directly from these limitations.

- **Multi-seed variance quantification.** This is the single most important deferred experiment. Repeating the full sweep across several seeds would put confidence intervals on every reported gap, confirm the robust findings (the eval-regime flip, the label-skew DP edge, the refusal of naive DP-

FedAvg), and settle whether the non-monotonic ε orderings on the Dirichlet partitions are real or single-seed artifacts.

- **Multi-label, twelve-lead PTB-XL.** The binary single-lead reduction was forced by the representational bottleneck of Lead II on the five-super-class task. Restoring all twelve leads and treating PTB-XL as a multi-label problem, with macro-AUC over the five super-classes as the metric, is the natural way to test whether the findings hold on a richer, clinically fuller task.
- **Architectural breadth.** Whether the eval-regime flip and the partition-dependent DP behavior persist under ResNet-style 1-D networks or Transformer encoders — architectures with different normalization footprints — would establish how much of the story is specific to a BatchNorm-heavy CNN.
- **Scale and realism of the federation.** Increasing the client count toward the dozens-to-hundreds of real deployments, and ideally validating on genuinely cross-institutional ECG data rather than simulated partitions, would test the findings under realistic covariate shift. Adopting a patient-disjoint, inter-patient evaluation split would further harden the central-evaluation numbers.
- **Stronger threat models.** The present threat model leaves the deployed personalized model’s BatchNorm statistics unprotected, on the grounds that they never cross the federation boundary. A setting in which an adversary can issue repeated queries against a deployed per-client model could in principle infer those statistics, and extending DP-FedBN to protect the deployed model — not only the federated updates — is an open problem worth addressing before clinical deployment.
- **Tuning and method breadth.** A proper sweep of the FedProx coefficient, and the inclusion of the FedBN extensions deliberately set aside here (batch-specific normalization, personalized data transformations), would situate DP-FedBN within a fuller personalization landscape under DP.

Taken together, these directions would move DP-FedBN from a carefully characterized single-seed construction on two ECG tasks toward a validated tool

for privacy-preserving, personalized medical federated learning. The construction itself — keep BatchNorm local, protect only what crosses the boundary, and report both evaluation regimes — is the contribution this thesis stands behind, in exactly those measured terms.

REFERENCES

- [1] Q. Li, B. He, and D. Song, “Adversarial collaborative learning on non-IID features,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 202, 2023, pp. 19504–19526.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 54, 2017, pp. 1273–1282.
- [3] N. Rieke, J. Hancox, W. Li, F. Milletari, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, *et al.*, “The future of digital health with federated learning,” *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.
- [4] R. C. Geyer, T. Klein, and M. Nabi, “Differentially private federated learning: A client level perspective,” in *NeurIPS Workshop on Machine Learning on the Phone and other Consumer Devices*, 2017.
- [5] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318.
- [6] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 37, 2015, pp. 448–456.
- [7] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, “Opacus: User-friendly differential privacy library in PyTorch,” *arXiv preprint arXiv:2109.12298*, 2021.

[8] Y. Wu and K. He, “Group normalization,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, vol. 11217. Springer, 2018, pp. 3–19.

[9] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, “FedBN: Federated learning on non-IID features via local batch normalization,” in *International Conference on Learning Representations (ICLR)*, 2021.

[10] Opacus contributors, “Allow batchnorm layers with frozen running stats (issue #472),” GitHub, 2022. [Online]. Available: <https://github.com/pytorch/opacus/issues/472>

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.

[12] T. Jia, L. Meng, S. Li, J. Liu, and D. Wu, “Federated motor imagery classification for privacy-preserving brain-computer interfaces,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 32, pp. 3442–3451, 2024.

[13] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, “PrivateFL: Accurate, differentially private federated learning via personalized data transformation,” in *32nd USENIX Security Symposium (USENIX Security)*. USENIX Association, 2023, pp. 1595–1612.

[14] P. Riedel, R. von Schwerin, D. Schaudt, A. Hafner, and C. Späte, “ResNetFed: Federated deep learning architecture for privacy-preserving pneumonia detection from COVID-19 chest radiographs,” *Journal of Healthcare Informatics Research*, vol. 7, no. 2, pp. 203–224, 2023.

[15] V. C. Marella, S. R. Veluru, and S. T. Erukude, “FedOnco-Bench: A reproducible benchmark for privacy-aware federated tumor segmentation with synthetic CT data,” 2025.

[16] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, *et al.*, “Advances

and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[17] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.

[18] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH arrhythmia database,” *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.

[19] ANSI/AAMI, “Testing and reporting performance results of cardiac rhythm and ST segment measurement algorithms,” ANSI/AAMI EC57:2012(R)2020, American National Standard, 2012.

[20] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic classification of heartbeats using ECG morphology and heartbeat interval features,” *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.

[21] V. Mondéjar-Guerra, J. Novo, J. Rouco, M. G. Penedo, and M. Ortega, “Heartbeat classification fusing temporal and morphological information of ECGs via ensemble of classifiers,” *Biomedical Signal Processing and Control*, vol. 47, pp. 41–48, 2019.

[22] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiselmeyer, F. I. Lunze, W. Samek, and T. Schaeffter, “PTB-XL, a large publicly available electrocardiography dataset,” *Scientific Data*, vol. 7, no. 1, p. 154, 2020.

[23] C. Dwork and A. Roth, “The algorithmic foundations of differential privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[24] I. Mironov, “Rényi differential privacy,” in *IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017, pp. 263–275.

[25] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, “Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network,” *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.

[26] M. Kachuee, S. Fazeli, and M. Sarrafzadeh, “ECG heartbeat classification: A deep transferable representation,” in *IEEE International Conference on Healthcare Informatics (ICHI)*, 2018, pp. 443–444.

[27] N. Strodthoff, P. Wagner, T. Schaeffter, and W. Samek, “Deep learning for ECG analysis: Benchmarks and insights from PTB-XL,” *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 5, pp. 1519–1528, 2021.

[28] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, T. N. Hoang, and Y. Khazaeni, “Bayesian nonparametric federated learning of neural networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 7252–7261.

[29] T.-M. H. Hsu, H. Qi, and M. Brown, “Measuring the effects of non-identical data distribution for federated visual classification,” *arXiv preprint arXiv:1909.06335*, 2019.

[30] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 3–18.

[31] L. Zhu, Z. Liu, and S. Han, “Deep leakage from gradients,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[32] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, “Learning differentially private recurrent language models,” in *International Conference on Learning Representations (ICLR)*, 2018.

[33] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.